

基于多模态时序分析与动态兴趣 建模预测社交媒体用户行为

目录

1 摘要	4
2 引言	4
3 问题介绍	4
3.1 问题重述	4
3.2 问题分析	5
3.3 模型假设	6
3.4 符号说明	6
4 预测用户 7 月 21 日行为	7
4.1 数据预处理	7
4.2 Fourier 级数拟合	8
4.3 预测 7 月 21 日的关注数据	8
5 预测用户 7 月 22 日博主关注情况	9
5.1 权重预处理	10
5.1.1 行为发生频率与分布	10
5.1.2 行为权重赋值的合理性分析	10
5.2 模型构建	11
5.2.1 数据基础与预处理	11
5.2.2 趣值量化规则	12
5.2.3 兴趣值动态衰减机制	12
5.3 可视化呈现	12
5.3.1 可视化参数的选择	13
5.4 模型求解	13
5.4.1 数据加载与预处理	13
5.4.2 预测机制	14

6	预测用户 7 月 21 日是否上线及互动博主排名	14
6.1	用户行为及心理分析	14
6.1.1	近期效应与新鲜度感知	14
6.1.2	活动频率与用户粘性	14
6.1.3	行为趋势动态演变	14
6.1.4	个体行为模式探究	15
6.2	模型判断机制	16
6.2.1	极端不活跃排除机制	16
6.2.2	极端活跃确认规则（优先判定为“上线”）	16
6.2.3	综合信号评分与阈值决策（处理中间状态用户）	16
6.2.4	特殊情况判定规则	17
6.3	模型求解	17
6.3.1	用户互动特征提取	17
6.3.2	互动潜力评分模型与排序机制	18
7	预测用户 7 月 23 日不同时段博主互动排名	19
8	模型评价与推广	21
8.1	傅里叶模型	21
8.1.1	动态用户群体	21
8.1.2	滑动窗口机制	21
8.2	一般化兴趣值函数模型	21
8.2.1	模型评价	21
8.2.2	模型推广	21
8.3	上线预测函数模型	23
8.3.1	模型评价	23
8.3.2	模型推广	23
8.4	互动潜力评分模型	24
8.4.1	模型评价	24

9 参考文献	24
10 附录	25
10.1 兴趣函数多博主图像绘制	25
10.2 绘制 Fourier 函数图像	32
10.3 绘制各博主逐小时互动频次表	36
10.4 检索代码	36
10.5 关注可能性函数	37
10.6 上线检测	43
10.7 互动数预测问题三	47
10.8 输入保存系统	52
10.9 问题三的兴趣判定	58

1 摘要

本论文针对社交媒体平台的用户行为分析与内容推荐优化问题，构建了一系列数学模型以预测用户行为并提升平台互动效率。我们基于 2024 年 7 月 11 日至 7 月 20 日和 7 月 22 日的用户与博主互动数据，分析并解决了博主的**新增关注数预测**、**用户新关注博主预测**、**用户在线行为及互动预测**、**用户在线时段与博主互动预测**四个问题。

在时序预测模型中，我们首先采用**离散傅里叶（Fourier）级数拟合方法**，观察博主新增关注数的周期性特征并进行平滑拟合，预测博主的总关注数。我们引入**马尔可夫链状态转移模型**预测用户单日在线概率，并结合**ARIMA 算法**分析用户上线和互动时段规律性，构建多模态时序-行为耦合模型，精确预测用户在特定时段的用户行为。

在兴趣与互动模型中，我们提出了一个用户-博主**兴趣度动态分析模型**。该模型通过赋予观看、点赞、评论等不同用户行为在不同时间下的**差异化权重**，量化某个用户对某个博主的兴趣程度，并引入**每日衰减机制**模拟兴趣随时间流逝而减弱的现象，便于合理预测用户的新关注行为。同时，我们建立了**多层次的判断机制**来预测用户是否在线，并基于加权互动数预测用户可能产生最高互动关系的三名博主。

我们基于历史数据，利用所构建的模型对 2024 年 7 月 21 日和 7 月 23 日的指定用户行为进行了预测，并验证了模型在实际应用中的有效性和对用户行为的洞察力。

最后，本研究通过结合统计分析、傅里叶变换、动态兴趣函数和多模态时序模型等多种数学建模方法，为社交媒体平台的用户行为预测和内容推荐优化提供了全面的解决方案。研究中考虑并改进了用户与博主数量固定、互动关系不变以及内容时效性等假设，为平台提升用户粘性和互动率提供了数据驱动的决策依据。

关键词：离散傅里叶级数拟合方法、马尔可夫链状态转移模型、ARIMA 算法、兴趣度动态分析模型、差异化权重、每日衰减机制、多层次判断机制

2 引言

在当今数字时代，社交媒体平台已成为人们日常生活中不可或缺的一部分，深刻影响着社会互动、信息获取与文化交流。这些平台通过构建多元化的线上交流空间，使得用户能够与基于专业知识或兴趣爱好的博主进行广泛互动，包括观看、点赞、评论乃至关注。博主则根据平台的推荐机制和用户反馈，持续调整并提升内容质量，以增强自身影响力；同时，用户的互动行为也反向塑造着平台的内容推荐系统。在此背景下，深入分析和预测用户与博主之间的互动行为关系，对于优化内容推荐策略、提升用户体验以及实现平台的可持续发展，具有举足轻重的意义。本研究聚焦于某社交媒体平台，深入分析已有的用户和博主之间的互动行为数据，进而预测用户行为并优化内容推荐方法。

3 问题介绍

3.1 问题重述

现某社交媒体平台需深入分析现有用户和博主之间的互动行为关系，来预测用户行为，并优化内容推荐方法。具体有以下四个问题：

问题 1: 新增关注数预测

基于 2024 年 7 月 11 日至 2024 年 7 月 20 日的用户行为数据, 分析用户与博主的历史互动模式 (如观看、点赞、评论、关注等). 利用这些数据预测 2024 年 7 月 21 日各博主的新增关注数, 并根据预测结果找出新增关注数最多的前五位博主 ID 及其新增关注数. 这一问题的目标是揭示用户的关注偏好和博主的吸引力, 帮助平台为博主提供精准的关注度预判.

问题 2: 用户新关注博主预测

根据 2024 年 7 月 22 日的用户行为数据 (观看、点赞、评论), 结合用户的历史互动行为, 建立模型预测哪些用户在该天可能进行新增关注, 并预测他们将关注哪些博主. 该问题旨在帮助平台根据用户的历史行为预测其新的兴趣点, 从而为其推荐适合的博主内容.

问题 3: 用户在线行为及互动预测

预测 2024 年 7 月 21 日某些指定用户是否在线, 若在线, 则进一步预测他们可能与哪些博主进行互动. 根据用户的历史数据, 模型需要识别出用户最有可能互动的前三名博主, 并预测他们的互动数量. 这有助于平台通过分析用户的在线行为和互动偏好, 为其推荐最有可能产生互动的博主, 从而提高平台的互动率和用户粘性.

问题 4: 用户在线时段与博主互动预测

在问题 3 的基础上, 进一步预测指定用户在 2024 年 7 月 23 日的在线状态, 并且分析每个用户在不同在线时段的行为特征. 预测每个时段与博主可能产生的互动数, 并为用户推荐互动数最高的三位博主及对应的时段. 这一问题侧重于根据用户的时间习惯, 精确预测用户在特定时段内的互动倾向, 以便平台更好地进行个性化推荐, 提高用户参与度.

3.2 问题分析

因为不考虑不同博主之间的相关性对同一用户的影响, 所以在进行数据处理时可以独立分析特定博主和特定用户之间的互动关系.

第 1 个问题预测用户 7 月 21 日行为, 需要我们通过历史的互动数据预测 7 月 21 日当天新增的关注数. 考虑到其结果受近期数据的影响更大, 我们采用了离散傅里叶分析方法, 对连续 72 小时 (3 天) 累积关注数据进行傅里叶级数拟合, 有效捕捉其周期性波动并平滑短期噪声. 随后, 利用最小二乘法对拟合结果进行缩放调整, 使其与近期实际数据趋势保持一致, 从而实现对未来新增关注数的精准外推预测. 这一方法可以有效揭示用户活跃度的日周期性.

第 2 个问题预测用户 7 月 22 日博主关注情况, 需要我们通过当天行为数据预测当天新关注的博主 ID. 为了准确描述该用户的行为特征, 我们构建了一个用户-博主兴趣度动态分析模型. 先引入一个由观看、点赞、评论这些行为数据生成的兴趣函数. 其中不同分量的占比根据过去一段时间用户的行为, 由大数据拟合确定. 并且, 这个函数值是随着时间逐渐积累的. 同时, 过程中我们还设置了衰减函数, 用户在没有新互动刺激时, 该衰减函数会让兴趣函数的值以预设的每日衰减速率持续降低. 之后, 我们再确定一个阈值, 当兴趣函数的值到达了 this 阈值, 用户就会进行关注. 关注行为被视为一种特殊的状态转换点, 此后兴趣值将不再衰减. 通过这样两个函数效果叠加, 最终确定当日的新增关注博主.

第 3 个问题预测用户 7 月 21 日是否上线及互动博主排名, 题目将互动数描述为点赞数、评论数、关注数的简单相加. 为了更符合真实情况, 我们在求和时进行了加权处理, 即变成了一个新的兴趣函数. 之后的想法与问题 2 基本一致, 预测的兴趣值若超过了阈值即为“高互动博主”. 于是, 我们设计了一个多层次判断机制. 具体来说, 首先, 引入极端不活跃排除机制, 快速识别并过滤掉长时间未登录或活跃度极低的用户. 其次, 设置极端活跃确认规则, 对于近期表现出极强活跃性的用户 (如连续多日上线或短期内登录频率极高), 直接判定为上线. 对于介于两者之间的“中间状态”用户, 我们则启动一个综合信号评分机制. 该机制从多个维度进行评估, 并对各项积极信号进行累积加分. 当总分超过预设阈值 (例如 6.5 分) 时, 预测用户上线, 否则默认预测为不上线.

第 4 个问题预测用户 7 月 22 日博主关注情况. 针对用户在线时段与博主互动预测, 我们构建了一个多模态时序-行为耦合模型. 该模型首先利用马尔可夫链状态转移模型预测用户单日在线概率, 综合考虑活跃天数与总天数之比、日均互动频次与最大互动频次之比, 并通过逻辑函数进行量化. 接着, 通过对近期 (如 7 月 19 日、20 日和 22 日) 数据加权的兴趣函数, 确定用户在预测日 (7 月 23 日) 的核心博主群体. 为了分析用户上线时段的规律性, 我们采用 ARIMA 算法对每天 24 个固定时段的活跃模式进行时间序列分析, 并利用雷达图进行可视化展示. 最终, 通过建立博主-时段互动频次关联矩阵并结合加权后的兴趣函数、历史点赞率和评论率等, 预测用户在每个在线时段与每个博主可能产生的互动数, 从而实现精准的、时段化的博主推荐.

3.3 模型假设

1. 该平台用户和博主数量固定, 不存在平台新用户/博主的加入和账号注销行为;
2. 用户和博主的互动关系建立后不再变化, 即平台中用户不存在取消点赞、删除评论、取消关注的行为;
3. 一位博主发布视频的时效性假定为 3 天, 即距离视频发布时间超过三天后该博主的互动增加量与该视频无关;
4. 不同博主之间视频的相关性 (即同类型视频推流) 忽略不计;
5. 只从数据的角度分析博主、用户行为, 不考虑现实可能存在的恶意刷评等行为.

3.4 符号说明

表 1: 符号说明

符号	符号说明
t, T	时间 (小时)
α	最小二乘法计算缩放因子
$DAILY_DECAY_AMOUNT$	该组目标任务量
$\Phi(\cdot)$	逻辑函数
α_1, β_1	加权系数
D_a, D_{total}	活跃天数
F_d, F_{total}	互动次数

4 预测用户 7 月 21 日行为

4.1 数据预处理

原始数据记录了用户与 42 位博主的四类互动行为与时间戳. 为了分析关注数的变化规律, 我们首先筛选并统计了所有 42 位博主逐小时的新增关注数. 基本结果如下图所示:

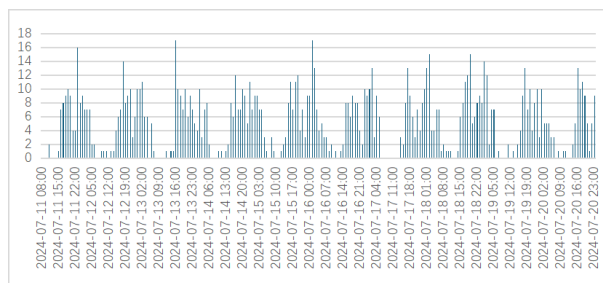


图 1: B16 新增关注情况

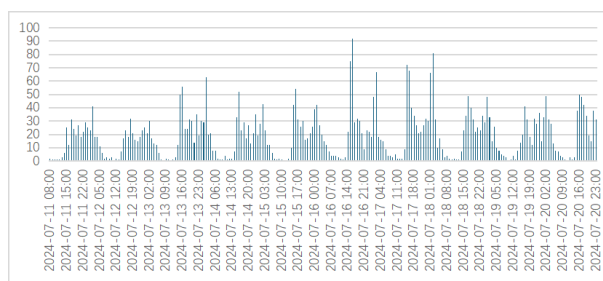


图 2: B21 新增关注情况

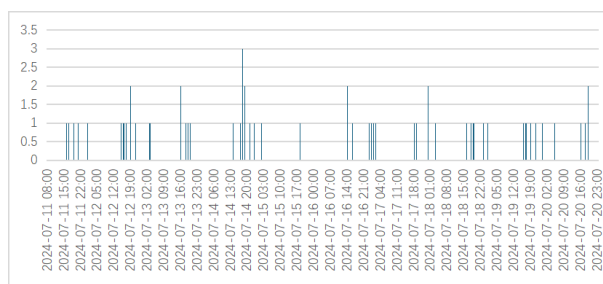


图 3: B9 新增关注情况

初步观察发现: 多数博主的新增关注数大体上呈现以 24 小时为周期的变化规律. 这表明用户活跃度或内容消费习惯可能存在日周期性.

以 B10 为代表的部分博主存在“断更”现象, 在某一天或若干天内, 其观看、点赞、评论、关注的各项数据均为 0. 这代表该博主在这些时段内没有发布新作品且其过去的作品在这段时间内没有用户观看. 这一现象暗示了博主发布作品后, 其内容的传播时间较为有限, 在较长的一段时间内一位博主的各种互动信息有随着作品发布时间以若干天为周期波动的现象.

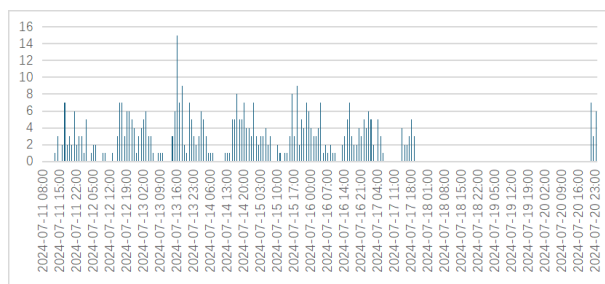


图 4: B10 新增关注情况

在观察按日统计的关注数变化时,考虑到博主发布作品后,其内容的传播和发酵需要一定时间,直接按日分析确实无法完全捕捉其影响周期.观察逐日统计数据的变化,基于内容传播的时间差的考虑,以每三天为一个分析周期进行后续统计预测.

4.2 Fourier 级数拟合

为了捕捉关注数变化的周期性特征并平滑短期波动,我们选择离散 Fourier 分析的方法.选取 2024 年 7 月 12 日至 7 月 20 日的数据共完整的 9 天为范围.首先统计出 7 组连续的 3 天数据段(7 月 12-14 日,7 月 13-15 日,……,7 月 18-20 日).将每个 3 天周期内的关注数进行累加,作为傅里叶分析的数据点,以获得更稳定的 72 小时 3 天的数据.

对 42 位博主的数据处理后的 3 天关注数数据分别进行了 $[0,72)$ 上的 72 阶傅里叶变换.一个周期为 T 的函数 $f(t)$ 其 N 阶傅里叶级数展开的一般形式为:

$$f(t) \approx \frac{a_0}{2} + \sum_{n=1}^N \left(a_n \cos\left(\frac{2\pi nt}{T}\right) + b_n \sin\left(\frac{2\pi nt}{T}\right) \right)$$

由 $T=72$, $N=72$, 带入可得每位博主关注数拟合出的 Fourier 函数:

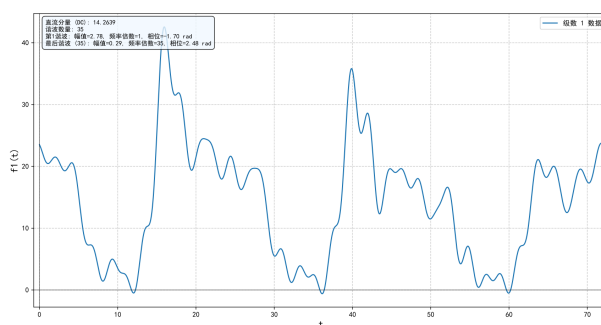


图 5: B10 的傅里叶级数

4.3 预测 7 月 21 日的关注数据

考虑到作品传播发酵的延后效应,由以上分析可以断言 7 月 21 日的关注数据主要由 7 月 19 日和 7 月 20 日的数据得到.从每位博主拟合的 72 小时傅里叶函数中,提取前 48 小时(即周期内前两天)的函

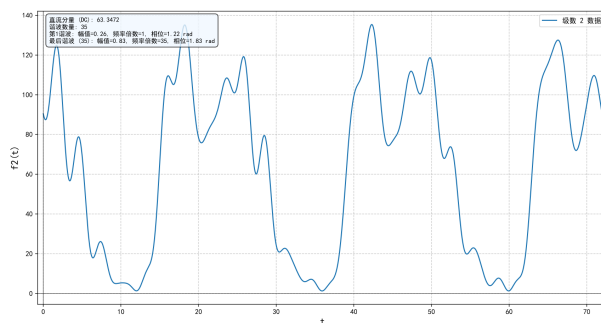


图 6: B12 的傅里叶级数

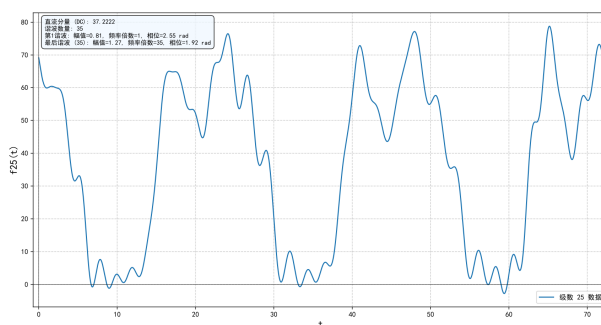


图 7: B45 的傅里叶级数

数值, 代表基于其历史 3 天周期的典型两日关注趋势. 将这个 48 小时的拟合结果与 7 月 19 日和 7 月 20 日的实际新增关注数进行对比. 通过最小二乘法计算缩放因子 α , 使预测值更贴近近期趋势.

目标函数为:

$$\min_{\alpha} \sum_{t=0}^{47} (y_{\text{实际}}(t) - \alpha \cdot f(t))^2$$

利用调整后的函数 $\alpha \cdot f(t)$ 外推至 7 月 21 日全天 ($t=48-71$), 即求得 7 月 21 日每位博主逐小时的预测新增关注数. 累加得到 7 月 21 日总预测新增关注数.

表 2: 用户 7 月 21 日行为结果

排名	1	2	3	4	5
博主 ID	B5	B21	B15	B13	B60
新增关注数	513	502	376	279	234

5 预测用户 7 月 22 日博主关注情况

在复杂的社交网络与内容平台中, 理解用户对博主的兴趣演变对于预测其未来与博主间互动行为有重大意义. 当然, 我们需要指出用户的兴趣并非一成不变, 它会随着时间的推移、互动行为而动态变化. 为了量化并追踪这种动态兴趣, 我们构建了一个用户-博主兴趣度动态分析模型. 该模型旨在根据用

户与博主之间随时间发生的具体互动行为，计算出一个动态的兴趣值，并通过可视化手段直观展示其变化趋势。

整个问题的思路可以由下述流程图体现：

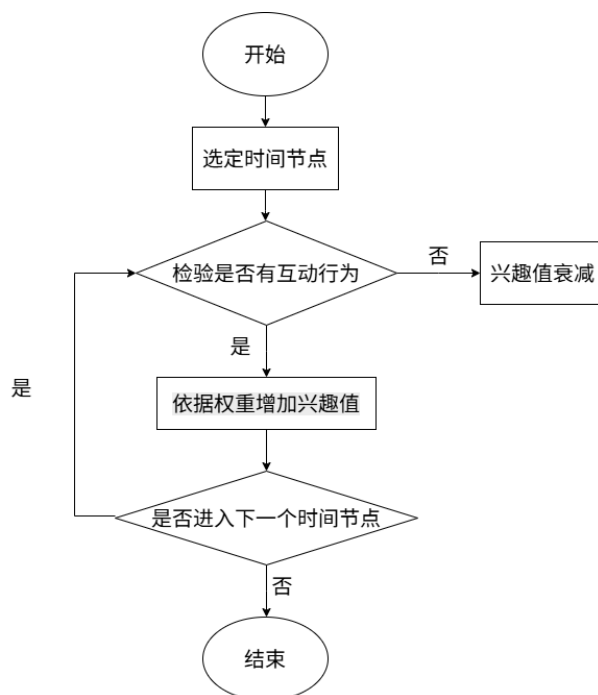


图 8: 预测用户 7 月 22 日博主关注情况流程图 1

5.1 权重预处理

上述提到的兴趣值由不同互动行为（观看、点赞、评论、关注）组成，每一种行为在其中应该占有不同的权重。因此，我们需要对数据进行初步的分析，选择尽量准确的权重来表示兴趣值。操作步骤如下：

5.1.1 行为发生频率与分布

从数据片段中观察，以用户 U1 为例，其与博主 B60、B21、B15 等的互动记录，我们不难发现行为代码“1”出现的频率是最高的。当然这也符合现实情况。

关于行为点赞与行为评论的出现频率相对较低，但必定伴随于观看行为。同时对绝大多数用户而言，点赞的数量显著多于评论，因此我们认为评论是兴趣度更加高的体现（无论是用户认可抑或是否定博主）。

5.1.2 行为权重赋值的合理性分析

观看是最普遍的互动形式，是用户接触内容的基础。赋予其 0.5 的基础增量，既能积累用户的初步兴趣，又不至于使其高频性而使兴趣值过早饱和，与现实情况出入较大，同时为后续更深层次互动留出空间。

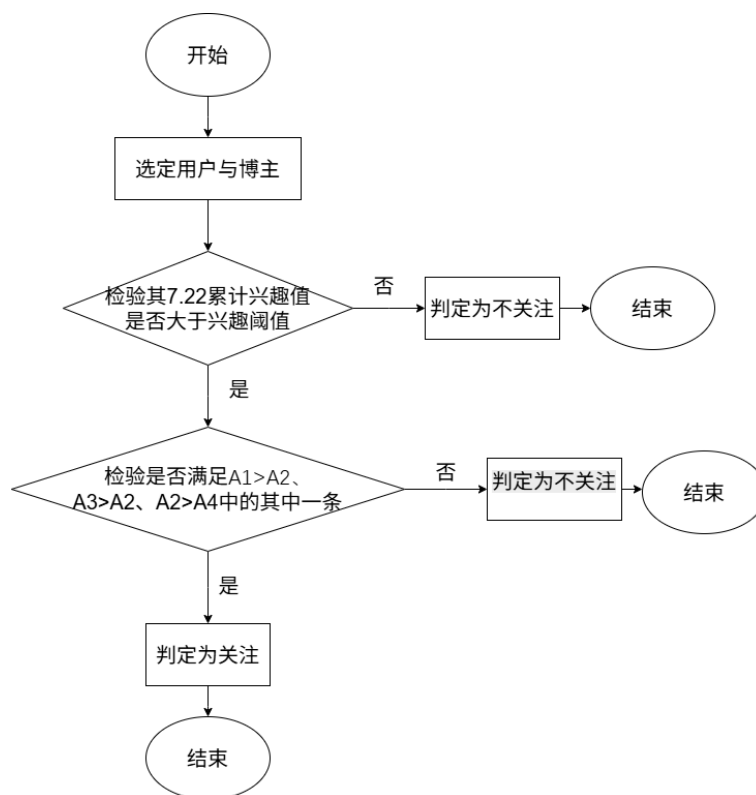


图 9: 预测用户 7 月 22 日博主关注情况流程图 2

经对网络互动行为较为丰富的用户的数据统计,我们发现用户平均观看 4-5 次,会出现一次点赞行为,故将点赞行为赋值为权重 2. 而评论的频率则较为无规律,其随用户的变化存在较大的差异. 但其无规律恰恰表明这是更加个性化、更深思熟虑的行为. 但从整体数据和用户行为常识来看,评论无疑比点赞需要更高的用户投入. 为了凸显这种差异,我们将“评论”的权重设定为 3.0.

我们将“关注”作为最高层次的兴趣度体现,但其并不体现在兴趣值上. 在用户关注博主后,两者间兴趣值函数将保持恒定. 实际上,该恒定值的设定并不会对函数功能造成影响.

5.2 模型构建

在上述基础上,我们提出构建一个以时间为横轴,用户对特定博主的兴趣程度为纵轴的兴趣函数 $I(t)$. 该函数的核心机制是: 用户的每一次积极互动都会提升其对该博主的兴趣值,而随着时间的自然流逝,若无新的互动刺激,这种兴趣会逐渐衰减.

5.2.1 数据基础与预处理

模型的数据来源于用户与博主间的互动日志,主要包含以下关键字段:

用户 ID: 唯一标识用户的字符串.

博主 ID: 唯一标识博主的字符串.

时间: 用户行为发生的时间节点,精确到分钟或秒.

用户行为：记录用户具体互动类型的编码（例如，1 代表观看，2 代表点赞，3 代表评论，4 代表关注等）。

在数据加载函数后，我们进行了必要的预处理：

统一列名：确保不同数据源的列名一致，方便后续处理。

时间格式转换：将原始时间字符串解析为 Pandas 的 Timestamp 对象，以便进行时间节点分析。

ID 类型统一：将用户 ID 和博主 ID 转换为字符串类型，确保比较和筛选的准确性。

数据合并与排序：将来自不同文件或时段的数据合并，并严格按照时间顺序对所有互动记录进行排序，这是保证兴趣度按时序正确累积和衰减的前提。

5.2.2 趣值量化规则

经过前面的数据分析阶段，根据用户行为对兴趣度的不同贡献程度，我们设定了如下的兴趣增量规则：

观看 (行为代码 1)：每次观看，兴趣值增加 0.5. 这代表了用户对内容的基础关注。

点赞 (行为代码 2)：每次点赞，兴趣值增加 2.0. 点赞通常表示比观看更深一层的认可。

评论 (行为代码 3)：每次评论，兴趣值增加 3.0. 评论是更积极的互动，通常表明用户投入了更多精力并愿意参与讨论。

关注行为 (行为代码 4)：关注行为是一个特殊的状态转换点. 在本模型的一个版本中，一旦用户关注了博主，其兴趣度可以被设定为一个较高的固定值

5.2.3 兴趣值动态衰减机制

现实生活中，记忆和兴趣会随时间衰减. 为了模拟这一现象，模型引入了每日衰减机制：

衰减条件：当用户与博主首次产生互动后，若后续没有新的互动行为，其累积的兴趣值将开始衰减。

衰减速率：兴趣值以每天固定的速率 *DAILY_DECAY_AMOUNT*（例如选取 0.5）进行衰减。

衰减计算：衰减按天计算. 如果距离上一次兴趣值更新（或衰减计算日）过去了 N 天，则总衰减量为 $N * DAILY_DECAY_AMOUNT$ 。

兴趣下限：兴趣值不会衰减至负数，其最小值为 0。

关注状态下的衰减：若模型设定关注后兴趣度固定，则在关注状态下，兴趣值不再进行衰减。

5.3 可视化呈现

为了直观地展示用户与博主之间兴趣值的动态变化，我们同时使用 matplotlib 库进行绘图。

以 U2642 用户为例，我们绘制了其关于两位不同博主的兴趣函数，展示如下：

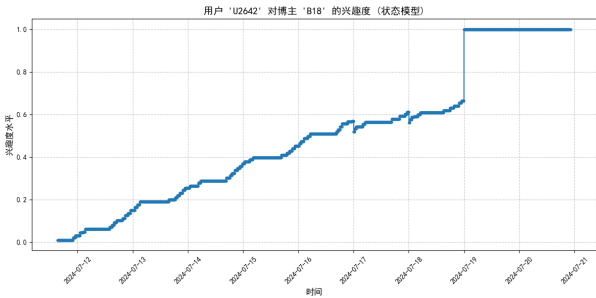


图 10: U2642 用户关于 B18 博主兴趣函数

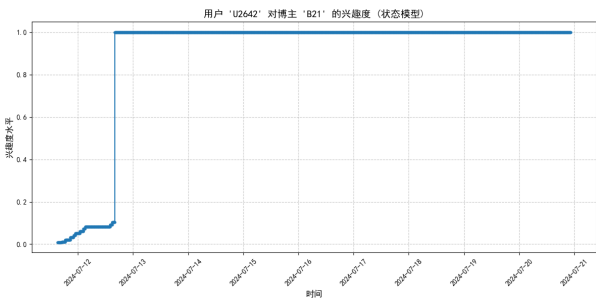


图 11: U2642 用户关于 B21 博主兴趣函数

5.3.1 可视化参数的选择

- 时间范围:** 用户可以通过输入起止时间来指定分析和绘图的具体时间段.
- 时间密度:** 用户可以调整绘图时横坐标的单位长度 (时间频率), 例如:
- '1D': 每天一个数据点, 适合观察长期趋势.
 - '1H': 每小时一个数据点, 适合观察日内变化.
 - '30min','15min','5min': 更小的时间密度设定, 适合观察短时间内由密集互动引起的兴趣波动.

5.4 模型求解

在建立一般化兴趣函数后, 我们可以对给出用户关注情况的预测.

5.4.1 数据加载与预处理

1. 自动读取 C1.csv (7.11–7.20 全量历史) 与 C2.csv (7.22 当天记录), 以及各用户单独 Excel (若存在).
2. 验证并清洗四个必要字段: 用户 ID、用户行为、博主 ID、时间.
3. 合并所有有效表格, 转换类型、排序、去空.

分析数据我们知道, 用户若关注了某个博主, 则该用户在当日一定观看了该博主的视频 (即有 “4”

必有“1”)。

5.4.2 预测机制

利用之前建立的一般化兴趣函数，我们可以得到五位用户历史上所有关注博主的关注前的累计兴趣值，在观察数据后，我们将这些累计兴趣值中的最小值乘以 60% 作为该用户的关注阈值。

同时，我们分别计算 7.22 当天 (A1)、7.18–20 三日均 (A2)、7.19–20+ 当日 (A3)、7.15–20 周期 (A4) 的日均兴趣；判定“当天峰值 > 前期均值”“前期“波动”“低谷”条件。

当选定用户与博主间兴趣值大于关注阈值且 $A1 > A2$, $A3 > A2$, $A2 > A4$ 中有一个发生时，我们断言该用户会关注此博主。

通过上述操作，我们最终得到了不同用户新关注的博主，结果如下：

表 3: 用户 7 月 22 日博主关注情况结果

用户 ID	U7	U6749	U5769	U14990	U52010
新关注博主 ID	B7, B27	B24, B17	B15, B24	B15, B24	B13, B7

6 预测用户 7 月 21 日是否上线及互动博主排名

整个问题的思路可以由下页中流程图体现：

6.1 用户行为及心理分析

在具体构建用户上线预测模型之前，我们首先对用户的上线行为模式及背后的心理驱动因素进行分析：

6.1.1 近期效应与新鲜度感知

用户的行为往往具有“近期效应”，即最近发生的行为对未来行为的指示性更强。一个刚刚上线过的用户，其再次上线的可能性通常高于一个长期未上线的用户。因此，我们应构建“最后一次登录距今天数”这一指标，它一定程度上可以反应用户活动新鲜度。

6.1.2 活动频率与用户粘性

用户的行为在特定时间段内（例如过去三天、七天或十四天）的上线天数，是衡量其活跃频率和平台粘性的重要尺度。高频上线通常意味着用户对平台有较强的依赖性，预示着其在未来短期内再次上线可能性较大。

6.1.3 行为趋势动态演变

用户的活跃度并非一成不变。通过比较用户在相邻两个时间周期（例如近几天与再往前几天）的上线频率，我们可以捕捉到其活跃度是正在变得更活跃，还是趋于沉寂，亦或是保持平稳，

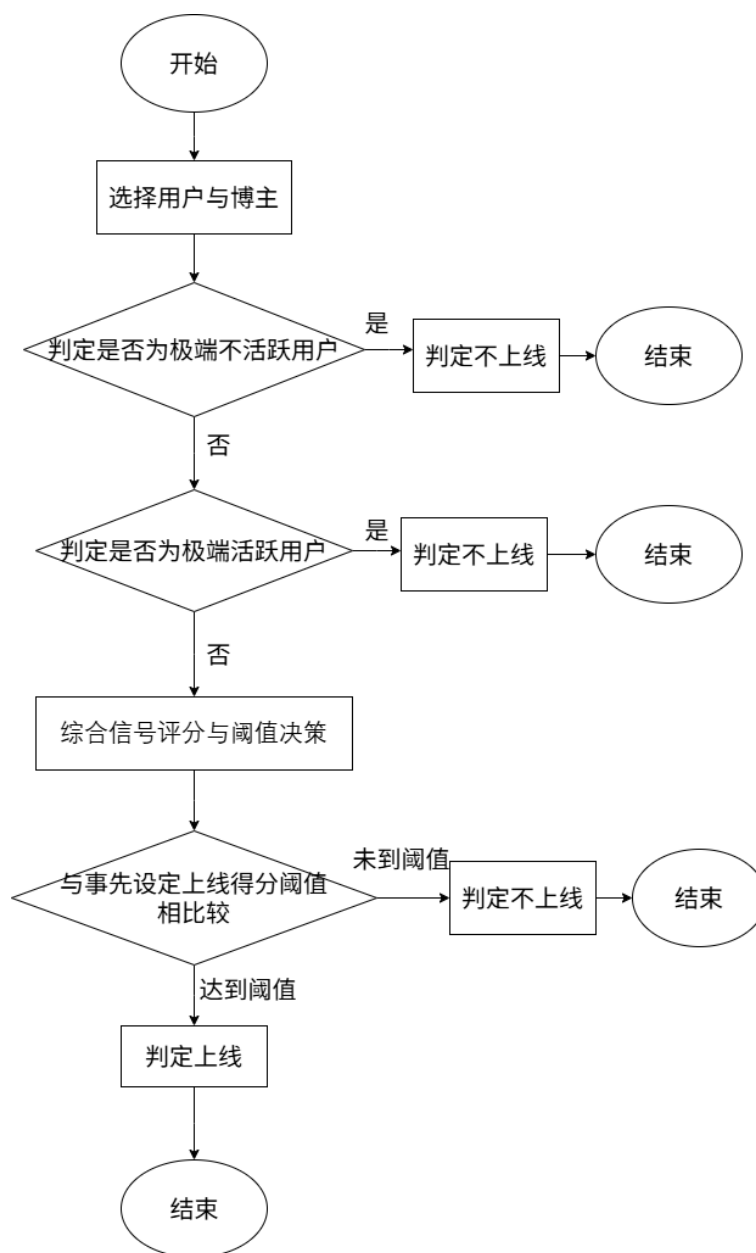


图 12: 预测用户 7 月 21 日是否上线及互动博主排名流程图

6.1.4 个体行为模式探究

部分用户可能形成了固定的上线周期, 例如习惯在每周的特定几天上线. 因此, 模型要考虑目标日期是否与用户历史上的活跃星期吻合度较高. 当然, 由于本题时间跨度较小, 故此项设置较低参照权重. 同时, 用户两次上线之间可能会存在一个相对稳定的时间间隔. 通过分析用户最近几次上线平均或中位数间隔, 我们可以判断当前未上线时长是否超出个体的习惯周期.

基于这些对用户行为及心理的分析, 我们明确了要提取的重要数据特征, 以进行后续具体的建模.

6.2 模型判断机制

6.2.1 极端不活跃排除机制

此机制旨在快速识别并排除那些行为特征显示其已明显处于持续不活跃状态的用户。触发条件包括但不限于：用户已长时间未登录（例如，超过十天）；或者用户在过去较长一段时间内（如十四天）的登录次数小于等于两次。且用户的行为特征满足此层面中的任何一项强不活跃条件，模型将直接预测该用户在目标日期“不上线”，并不再进行后续的判断流程。

6.2.2 极端活跃确认规则（优先判定为“上线”）

在排除了极端不活跃用户后，此层面致力于快速识别并确认那些近期行为表现出极强活跃性的用户。触发条件包括：用户在过去极短时间内（如三天）保持每日上线的高频率；或者用户在近期（如过去七天）的登录频率非常高（大于 5 次），并且其最后一次登录行为就发生在一两天之内。如上条件一旦发生，则直接判定为 7.23 用户会登录。

6.2.3 综合信号评分与阈值决策（处理中间状态用户）

对于行为特征既不属于极端不活跃也不属于极端活跃的“中间状态”用户，模型将启动一个综合信号评分机制。此机制会从多个维度评估用户的上线倾向：用户的“近期性”（最后一次登录离目标日期越近，得分越高）、“近期频率”（过去数天内登录次数越多，得分越高）、“活跃趋势”（趋势向上或平稳则加分）、“周期性吻合度”（如果目标日期恰好是用户历史上经常上线的星期几，则加分）以及“上线间隔规律性”（如果用户当前的未上线时长并未超出其通常的上线间隔，则加分）。模型会将这些积极信号进行累积评分，详细规则如下：

• 近期性

若用户在过去两天内（含两天）有登录记录，此项加 3 分。

若用户在过去三至四天内有登录记录，此项加 1 分。

若用户最后一次登录在四天以前，此项不加分。

近期频率（短期内的上线密度）：评估用户近期的活跃强度。

• 近期频率

i. 过去三天内登录频率：

若登录两次，此项加 3 分。显示出非常高的短期活跃度。

若登录一次，此项加 1.5 分。表明近期至少有过一次接触。

其余情况不加分。

ii. 过去七天内登录频率：

若登录三次或以上，此项加 2 分。反映了较好的一周内活跃水平。

若登录两次，此项加 1 分。表明一周内有一定活跃度。

其余情况不加分。

• 活跃趋势

若“近期活跃趋势比率”（最近五天登录次数 / 再往前五天登录次数）大于 1.2，表明活跃趋势明显向好，此项加 2 分。

若该比率在 0.8 到 1.2 之间（含 0.8 和 1.2），表明活跃趋势相对平稳或有小幅波动，此项加 1 分。

若该比率小于 0.8，表明活跃趋势向下，此项不加分。

• 周期性吻合度

假设用户在所给数据能呈现出的中一周中某日登录次数达到两次或以上，表明有一定的在该星期几上线的习惯，此项加 0.5 分。

• 上线间隔规律性

这用来评估用户是否“按惯例”到了该上线或该休息的时候。若用户的“中位数上线间隔”有效（即有足够历史数据计算得出），并且当前距离上次登录的天数小于或等于其中位数上线间隔加一天，这表明用户可能还未达到其通常的“休息”周期，或者正处于活跃期内，此项加 1 分。

当用户所得分数大于 6.5 分时，我们预测其上线。

6.2.4 特殊情况判定规则

如果用户历经上述三个层面的判断后，仍未能被明确划分为“上线”或“不上线”，则模型将启用默认判定规则。在此机制下，出于对预测准确性的考虑，模型将默认预测此类用户在目标日期“不上线”。这体现了一种相对保守的预测倾向，即在缺乏足够充分的积极上线信号时，模型倾向于认为用户将维持其当前的（相对不活跃或不确定）状态。

6.3 模型求解

我们使用前述上线预测模型，得到题目中的四位用户 U9，U22405，U16，U48420 均会在 7.23 上线，下面我们再度建立模型以预测可能产生互动数最高的博主。

首先，我们仍沿用问题二构建的兴趣值函数的思路，只不过此时此模型的核心目标并非仅仅预测单次关注行为，而是通过综合评估用户与各位历史互动博主之间的“互动潜力得分”，筛选出未来互动热度最高的前三位博主。且此时观看、点赞、评论、关注权重相等，均记作 1。

6.3.1 用户互动特征提取

在进行前述的数据预处理后，我们针对每一对“用户-博主”组合，提取了能够反映其互动深度、频率、新鲜度及周期性模式的关键特征：

i. 近期互动强度

短期高频互动：计算用户在目标预测日（例如七月二十三日）之前的极短窗口期内（如过去三天）与特定博主的总互动次数。这里的“互动次数”严格遵循本次预测任务的特殊设定，即观看、点赞、评

论、关注四种行为权重相等，均计为一次互动。

中期互动频率：同样计算用户在稍长一些的近期窗口（如过去七天）内与特定博主的总互动次数。

ii. 历史互动积累

历史总互动量：统计用户自最早记录，直至目标预测日前一天，与特定博主的全部互动行为总次数。

iii. 互动新鲜度

最后互动间隔：计算目标预测日前，用户距离其与特定博主最后一次互动已经过去的天数。

6.3.2 互动潜力评分模型与排序机制

基于上述提取的特征，我们为每一对“用户-候选博主”构建了一个动态的“互动潜力评分模型”。该模型旨在量化用户在目标日期与该博主发生多次互动的可能性。

1. 候选博主筛选

对于每个目标预测用户（如 U16、U48420），模型首先会从其完整的历史互动记录中，筛选出所有曾与之产生过互动的博主，构成该用户的“候选博主集合”。

2. 互动潜力评分细则

针对用户与每一位候选博主的组合，模型会根据以下细则计算其在目标预测日的互动潜力得分：近期高频互动加权：对过去三天内的短期互动总次数赋予较高的权重 3.0，而对过去七天内的中期互动总次数赋予次高权重 1.5。

历史互动总量优化加权：在实际运行过程中，我们发现若将历史互动总量所反应的互动潜力设定权重过大，则极可能导致其他评分细则难以发挥作用，最后导致模型的适用性变差。因此我们此处将总互动次数进行对数转换，以降低其值较大带来的影响。

互动新鲜度激励：根据距离上次互动的天数给予显著的加分。例如，若一天内刚互动过，则加 5 分；三日内加 3 分；七日内加 1 分。互动越新鲜，表明用户当前的兴趣点越可能集中在该博主身上。

3. 模型输出：潜力最高的前三名博主

在计算完用户与所有候选博主的互动潜力得分后，模型会对这些得分进行降序排列。最终，选取总得分最高的前三位博主作为该用户在目标预测日最有可能产生较多互动的对象。同时，模型也会输出这三位博主的具体潜力得分，以便进行相对比较和进一步分析。

通过上述步骤，我们计算得到了不同用户关注博主的潜力，并依此确定了最可能关注的三为博主：

表 4: 用户 7 月 21 日上线及互动博主排名结果

用户 ID	U9	U22405	U16	U48420
博主 ID 1	B13	B47	B24	B5
博主 ID 2	B8	B59	B6	B21
博主 ID 3	B7	B44	B2	B15

7 预测用户 7 月 23 日不同时段博主互动排名

针对用户行为与时段化互动预测问题, 我们构建了多模态时序-行为耦合模型. 以 U26447 用户为例, 首先基于历史数据统计活跃天数、日均互动频次构建时间序列模型. 引入马尔可夫链状态转移模型预测单日在线概率, 如 U26447 用户活跃天数 10 天, 仅 7 月 17 日未上线; 日均互动频次 33 次, 计算得到 U26447 在线概率超过 99%. 在线概率大于 50% 则认为其会上线.

$$P(Online) = \Phi(\alpha_1 \cdot \frac{D_a}{D_{total}} + \beta_1 \cdot \frac{F_d}{F_{max}})$$

其中 $\Phi(\cdot)$ 为逻辑函数, α_1, β_1 为加权系数. $D_a=10$, $D_{total}=11$, $F_d=33$, $F_{total}=74$

通过对 19 日、20 日和 22 日提高加权的兴趣函数确定用户在 23 日的核心博主群体, 认为这部分博主是大概率在 23 日会发生互动的博主. 并建立时间概率函数, 以量化比较互动概率. 提取出 B13、B45、B59、B44、B47 五位博主作为核心博主群体, 认为该用户在 23 日有 95% 以上的概率会与这些博主互动.

采用 ARIMA 算法分析用户上线时段规律性. 按 24 小时将每天划分为固定时段 (0:00-1:00, ..., 23:00-24:00) 通过雷达图分析历史时段活跃模式.

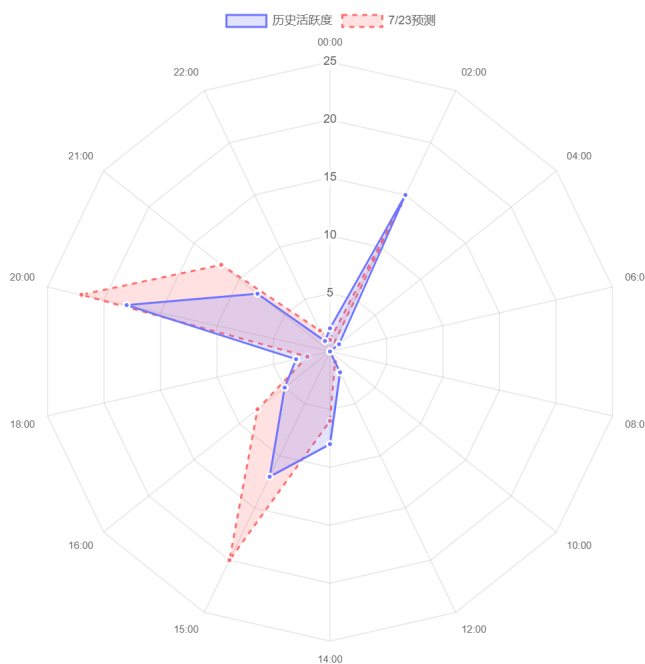


图 13: U26477 用户活跃时间雷达图

建立博主-时段按互动频次的关联矩阵, 提高 19 日、20 日、22 日的数据权重, 得到预测 23 日时间活跃度雷达图. 如 U26447 用户固定时段活跃为深夜 2 点、下午 15 点和晚上 20 点.

对于具体用户行为, 分析该用户对各位博主点赞率、评论率等的历史数据以及该用户关注的频率. 结合加权后的兴趣函数, 预测该用户在 23 日整体的互动情况.

表 5: U26477 在 7 月 23 日与博主互动情况预测表

	B10	B12	B13	B27	B35	B42	B43	B44	B45	B46	B47	B52	B53	B59	B7
2024/7/23 0:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 1:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 2:00	0	0	3.5	0	0	0	0	0	0	0	0	2.5	0	2.5	0
2024/7/23 3:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 4:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 5:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 6:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 7:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 8:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 9:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 10:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 11:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 12:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 13:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 14:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 15:00	0	0	0	0	0	2.5	2.5	0	2.5	0	0	0	0	0	0
2024/7/23 16:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 17:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 18:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 19:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 20:00	0	0	2.5	0	0	0	0	2.5	3.5	0	2.5	0	0	0	0
2024/7/23 21:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 22:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2024/7/23 23:00	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

由此，对多位用户进行相同的处理，我们得到了该问题的结果，如下：

表 6: 用户 7 月 23 日不同时段博主互动排名结果

用户 ID	U10	U1951	U1833	U26447
博主 ID1	B13	B5	B35	B13
时段 1	02:00-03:00	01:00-02:00	11:00-12:00	02:00-03:00
博主 ID2	B4	B23	B27	B45
时段 2	15: 00-16:00	04:00-05:00	18:00-19:00	20:00-21:00
博主 ID3	B12	B21	B10	B45
时段 3	15: 00-16:00	16:00-17:00	18:00-19:00	15:00-16:00

8 模型评价与推广

在解决本社交平台用户行为规律相关问题时，共建立了五个模型；关于预测用户在 7 月 21 日的行为时，我们建立了每个博主以三天为单位逐小时关注数 72 阶傅里叶级数相关模型；在预测指定用户 7 月 22 日关注情况时，我们建立了兴趣值函数；在预测用户 7 月 21 日是否上线时，我们建立了上线预测函数，同时在解决用户 7 月 23 日互动排名问题中，我们建立了思路类似于一般化兴趣值函数的互动潜力评分模型。

下面针对不同的模型进行评价与推广：

8.1 傅里叶模型

8.1.1 动态用户群体

在本题要求下，用户数量与博主数量是固定的。倘若取消这一限制，当有更多新用户加入时，可以引入新用户冷启动机制。例如，通过协同过滤算法，将新用户行为与相似历史用户比照，初始化其互动周期函数。这可以大大优化面向新用户的内容推荐机制。

8.1.2 滑动窗口机制

当前模型依赖固定历史周期（3 天），即单一视频互动数量的更新周期。实际应用中，可通过评估某位博主的作品传播周期设计动态滑动窗口，根据博主活跃度自适应调整窗口长度。如高活跃博主用 7 天窗口捕捉趋势，低活跃博主用 14 天窗口平滑噪声。

8.2 一般化兴趣值函数模型

8.2.1 模型评价

首先，兴趣值函数采取多行为差异性建模方式，对三种不同的互动行为采取不同的权重设定，以真实反应用户的兴趣程度。同时我们并非随意设定权重值，而是通过数据预处理以及详细的数据分析得到合理权重，这确保了模型的实用性。

其次，兴趣值评价函数有动态演化机制，通过细致的数据处理以及数据加载函数的设定，我们确保了在后续模型使用过程中，其时间步长可在相当大的范围内自由调整。保证了模型既可以预测半小时内用户详细的互动行为，又可以预测长达十天的用户整体互动趋势。大大增强了模型在不同场景下的适用性。

最后，该模型引入了兴趣衰减机制，使用户对博主的兴趣程度得到更合理的体现。同时，兴趣衰减函数可以自行设定，这一点保证了模型可适应用户心理研究及数据统计分析的变化。

但是，其权重的选择会受到个体影响。例如，有些人在观看视频时较容易点赞，而有些人对于点赞则较为慎重；有些人看完视频后对评论较为谨慎，有的人则喜欢在博主视频评论区中高谈阔论。

8.2.2 模型推广

首先，该模型将各类行为赋予不同的权重值；因此当引入同样在社交媒体中常见的取关、拉黑、私信等互动行为时，通过相应数据分析及用户行为洞察仍可设定合适的权重，确保模型的适用性！

其次，该模型不依赖于时间的长短，其算法和可视化过程都较为简洁清晰. 因此当所研究时间跨度增大时，其适用性不会降低.

下面给出使用实例：

表 7: 一般化兴趣函数模型的推广实例表

用户 ID (User ID)	用户行为 (User behaviour)	博主 ID (Blogger ID)	时间 (Time)
U284	-2	B5	2024/7/22 9:11
U284	1	B5	2024/7/22 9:15
U284	2	B12	2024/7/22 9:20
U284	-2	B12	2024/7/22 9:25
U284	3	B7	2024/7/22 9:30
U284	1	B7	2024/7/22 9:35
U284	-1	B23	2024/7/22 10:00
U284	2	B23	2024/7/22 10:05
U284	3	B45	2024/7/22 10:15
U284	-4	B45	2024/7/22 10:20
U284	1	B31	2024/7/22 10:45
U284	2	B31	2024/7/22 10:50

在此推广中，我们随机生成了 U284 的更多数据，其中“1”为观看，“2”为点赞，“3”为评论，“4”为关注，“-4”为拉黑,” -2”为去取关.

权重除观看为 0.5 外，其余均与数值相同.

在此表格数据下，我们的兴趣函数可视化如下所示：

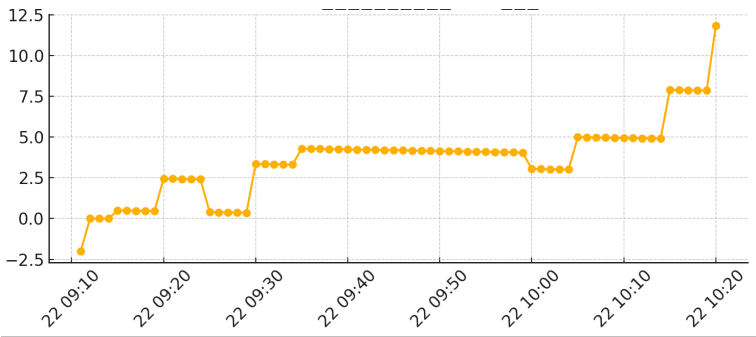


图 14: 互动行为多元化的可视化兴趣值折线图

其中，横、纵坐标分别表示时间和兴趣值.

8.3 上线预测函数模型

8.3.1 模型评价

上线预测函数同样是在深度研究用户行为后提取反应用户上线概率的重要特征，并对其进行机制设定。该模型的一个显著优势是：考虑到了互动新鲜度以及互动趋势，将用户最后一次互动距离 7.23 的时间长短考虑在内，同时将近些天互动量与前些天互动量的比值纳入考虑范围。这两点使模型不单是通过互动的线性增长预测 7.23 互动量最高的博主，而是多维度考虑互动情况的变化，给出更加贴近现实和用户心理的预测。

该模型另一个优势是其对用户个体行为模式的精细化考量与适应性。模型不仅分析了用户整体的近期活跃度，还特别关注了用户针对特定博主的“上线间隔规律性”以及“周期性互动习惯”。通过计算用户历史上与某博主互动的平均或中位数间隔，模型可以判断当前未互动时长是否已经超出了该用户的“沉默期”，从而更准确地评估其再次互动的可能性。此外，模型还会考察目标预测日是否与用户历史上和特定博主互动较为频繁的“星期几”相吻合，这种对个体周期性偏好的尊重，使得预测更具个性化和针对性。这种多维度、个体化的特征提取与综合评估，使得模型能够更好地区分不同用户的行为差异，避免“一刀切”的判断，从而提升了预测的精准度和场景适应性。

该模型的优势决定了其基本数据越多，结果越准确。但是模型也存在劣势。对于互动不积极的用户，该模型预测偏差较大。

其次，虽然规则和阈值可以调整，但在一次部署后它们通常是相对固定的。用户的行为模式可能会随时间演变，固定的规则可能无法完全适应这种动态变化，除非进行定期的模型校准和更新。

8.3.2 模型推广

实际上不仅仅用户心理会影响其行为，当外部因子出现时，例如节假日等因素，均会对用户行为产生影响。

为此，我们同样生成大量数据，给出节假日时相关预测：

对于模拟中春节期间互动可能增加的用户（如 U16）：

在评估近期登录天数时，达到“上线”判定所需的阈值可以适度降低。例如，平日可能需要过去 3 天登录 2 次才算强活跃，春节期间可能登录 1 次就可以视作积极信号，赋予与原函数相同的权重。

对于近期活跃趋势比率，即使比率没有显著上升（例如接近 1），但如果绝对登录天数符合节日期间的预期（例如，U16 平日登录少，春节期间因娱乐需求登录增多，即使趋势比率变化不大，也应视为积极信号）。

最后一次登录距今天的时间长短的容忍度可以略微放宽，因为节日期间的登录可能不如工作日规律。

对于模拟中春节期间互动可能减少的用户（如 U48420）：

相应的，触发极端不活跃用户的阈值可以收紧（例如，平日超过 10 天不上线才判断为不活跃，春节期间可能超过 5-7 天就认为其大概率因节日事务繁忙而不会上线）。

即使近期活跃度等指标在春节期间有所下降, 如果其下降幅度在预设的“节日效应”(例如 U48420 互动减少 20%-50%) 范围内, 模型不应立即将其判定为“活跃度趋势显著下降”从而触发强不活跃信号.

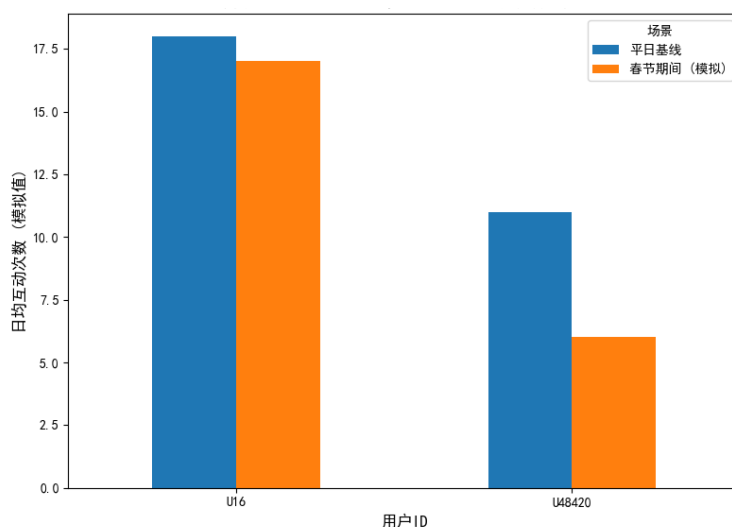


图 15: 平日与春节期间用户日均互动次数对比模拟图

8.4 互动潜力评分模型

8.4.1 模型评价

我们构建的这套旨在预测用户未来互动的关系. 通过对历史总互动量采用对数平滑处理, 避免了仅仅因为历史互动极多的“热点博主”的模型, 其核心优势首先体现在其对用户行为动态性的深度捕捉. 模型充分考虑了互动强度和互动新鲜度的模型, 两者均衡考虑, 确保模型运行机制符合现实情况.

但该模型仍有待改进之处. 其一, 对于互动记录稀疏或者行为模式极端不规律的用户, 模型难以提取到足够稳定的特征进行准确的潜力评分, 预测结果的可信度可能会有所下降.

其二, 模型采用用户行为权重相等的假设, 这一点并不利于反应真实的互动潜力. 评论和点赞的互动效应影响是显著高于观看的.

9 参考文献

- [1] 程慧平, 闻心王月. 社交媒体情境下用户信息行为研究 [M]. 北京: 中国社会科学出版社, 2024: 293.
- [2] 卢新元. 社交网络中用户行为研究: 积极与消极双重视角 [M]. 北京: 科学出版社, 2025: 298.

10 附录

10.1 兴趣函数多博主图像绘制

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from typing import Dict, Optional, Union
5 from reportlab.lib.pagesizes import letter
6 from reportlab.pdfgen import canvas
7
8 # --- 配置matplotlib支持中文 ---
9 plt.rcParams["font.family"] = ["SimHei", "sans-serif"]
10 plt.rcParams["axes.unicode_minus"] = False # 正确显示负号
11
12 # --- 配置参数 ---
13 INTEREST_INCREMENTS: Dict[int, float] = {
14     1: 0.5, # 例如: 点赞
15     2: 2.0, # 例如: 评论
16     3: 3.0, # 例如: 收藏/转发
17     # 用户行为代码 4 被特殊处理为关注
18 }
19 FOLLOW_ACTION_CODE: int = 4
20 FOLLOW_INTEREST_LEVEL: float = 1.0*15 # 关注后的固定兴趣度
21 DAILY_DECAY_AMOUNT: float = 0.05*0 # 每日衰减量 (未关注时)
22 DEFAULT_INCREMENT: float = 0.0 # 未知行为的默认增量
23
24
25 # --- 数据加载函数 ---
26 def load_data(c1_path: str = 'C1.csv', c2_path: str = 'C2.csv') -> pd.DataFrame:
27     try:
28         # 使用 'utf-8-sig' 来正确处理带有BOM的UTF-8文件
29         df1 = pd.read_csv(c1_path, parse_dates=['时间 (Time)'], encoding='utf-8-sig')
30         df2 = pd.read_csv(c2_path, parse_dates=['时间 (Time)'], encoding='utf-8-sig')
31         print(f"从 {c1_path} 和 {c2_path} 加载数据成功。")
32
33     except FileNotFoundError as e:
34         print(f"错误: {e}。请确保CSV文件 '{c1_path}' 和 '{c2_path}' 与脚本在同一目录下, 或者提供正确路
           ↪ 径。")
35         raise
36     except ValueError as e:
37         # 捕获因 '时间' 列不存在或无法解析而引发的 ValueError
38         if "Missing column provided to 'parse_dates'" in str(e) or "'时间' not in columns" in str(
39             e) or "could not be parsed" in str(e):
40             print(
41                 f"读取CSV文件时出错: 找不到名为 '时间' 的列或 '时间' 列无法被解析为日期。请检查CSV文件
           ↪ 中的列名和日期格式是否正确, 并与代码中的 '时间' 匹配。")
42             # 尝试打印列名以帮助调试
43             try:
44                 temp_df1_cols = pd.read_csv(c1_path, encoding='utf-8-sig', nrows=0).columns.tolist()
45                 print(f"文件 {c1_path} 的检测到列名: {temp_df1_cols}")
46             except Exception as read_err:
47                 print(f"尝试读取 {c1_path} 列名失败: {read_err}")
48             try:
49                 temp_df2_cols = pd.read_csv(c2_path, encoding='utf-8-sig', nrows=0).columns.tolist()
```

```

50         print(f"文件 {c2_path} 的检测到列名: {temp_df2_cols}")
51     except Exception as read_err:
52         print(f"尝试读取 {c2_path} 列名失败: {read_err}")
53     else:
54         print(f"读取CSV文件时发生值错误: {e}")
55         raise
56 except Exception as e:
57     print(f"读取CSV文件时发生未知错误: {e}")
58     raise
59
60 required_cols = ['用户ID (User ID)', '博主ID (Blogger ID)', '时间 (Time)', '用户行为 (User
    ↳ behaviour)']
61 for df_check, name in [(df1, c1_path), (df2, c2_path)]:
62     # 去除列名中可能存在的前后空格
63     df_check.columns = df_check.columns.str.strip()
64     for col in required_cols:
65         if col not in df_check.columns:
66             actual_cols_message = f"文件 {name} 的实际列名为: {list(df_check.columns)}" if not
    ↳ df_check.empty else f"文件 {name} 为空或无法读取列名。"
67             raise ValueError(
68                 f"文件 {name} 中缺少必需的列 '{col}' (期望的列名列表是 {required_cols})。\\n{
    ↳ actual_cols_message}")
69
70 # 确保ID列是字符串类型, 以便与输入的字符串ID进行比较
71 df1['用户ID (User ID)'] = df1['用户ID (User ID)'].astype(str)
72 df1['博主ID (Blogger ID)'] = df1['博主ID (Blogger ID)'].astype(str)
73 df2['用户ID (User ID)'] = df2['用户ID (User ID)'].astype(str)
74 df2['博主ID (Blogger ID)'] = df2['博主ID (Blogger ID)'].astype(str)
75
76 # df_all = pd.concat([df1, df2], ignore_index=True)
77 df_all = pd.concat([df1], ignore_index=True)
78 return df_all
79
80
81 # --- 新的兴趣度计算函数 ---
82 def compute_interest_series_new_model(
83     df: pd.DataFrame,
84     user_id: str, # 明确ID为字符串类型
85     up_id: str, # 明确ID为字符串类型
86     start_time_requested: Optional[pd.Timestamp] = None,
87     end_time_requested: Optional[pd.Timestamp] = None,
88     freq: str = '1H' # 输出频率
89 ) -> pd.Series:
90     print(f"compute_interest_series_new_model: 正在为用户 '{user_id}', 博主 '{up_id}' 筛选数据...")
91     # 确保DataFrame中的ID列也是字符串类型, 以便正确比较
92     # (已在load_data中处理, 但双重检查无害或在此处再次转换)
93     # df['用户ID'] = df['用户ID'].astype(str)
94     # df['博主ID'] = df['博主ID'].astype(str)
95
96     subset = df[(df['用户ID (User ID)'] == user_id) & (df['博主ID (Blogger ID)'] == up_id)].sort_values
    ↳ (by='时间 (Time)')
97     print(f"compute_interest_series_new_model: 筛选后数据条数: {len(subset)}")
98
99     total_points = 0
100    for _, event in subset.iterrows():

```

```

101     increment = INTEREST_INCREMENTS.get(event['用户行为 (User behaviour)'], DEFAULT_INCREMENT)
102     total_points += increment
103     print(f"用户 '{user_id}' 与博主 '{up_id}' 所有互动的积分之和: {total_points}")
104
105     if subset.empty:
106         print(f"未找到用户 '{user_id}' 和博主 '{up_id}' 之间的交互记录。")
107         if start_time_requested and end_time_requested:
108             time_index_empty = pd.date_range(start=start_time_requested, end=end_time_requested, freq=
109                 ↪ freq)
110             if time_index_empty.empty and start_time_requested == end_time_requested:
111                 time_index_empty = pd.date_range(start=start_time_requested, periods=1, freq=freq)
112             return pd.Series(0.0, index=time_index_empty, name='Interest')
113         return pd.Series(dtype=float, name='Interest')
114
115     sim_start_time = start_time_requested if start_time_requested is not None else subset['时间 (Time)'.
116         ↪ ].min()
117     sim_end_time = end_time_requested if end_time_requested is not None else subset['时间 (Time)'].max
118         ↪ ()
119
120     if sim_start_time == sim_end_time:
121         sim_end_time = sim_start_time + pd.to_timedelta(freq)
122
123     effective_start_for_range = sim_start_time
124     effective_end_for_range = sim_end_time
125
126     output_time_index = pd.date_range(start=effective_start_for_range, end=effective_end_for_range,
127         ↪ freq=freq)
128
129     if output_time_index.empty:
130         if effective_start_for_range == effective_end_for_range:
131             output_time_index = pd.date_range(start=effective_start_for_range, periods=1, freq=freq)
132
133     if output_time_index.empty:
134         print(
135             f"警告: 生成的 output_time_index 为空。起始: {effective_start_for_range}, 结束: {
136                 ↪ effective_end_for_range}, 频率: {freq}")
137         return pd.Series(dtype=float, name='Interest')
138
139     final_values = np.zeros(len(output_time_index))
140     current_interest_final: float = 0.0
141     has_followed_final: bool = False
142     last_decay_date_final: Optional[pd.Timestamp.date] = None
143
144     actual_follow_time: Optional[pd.Timestamp] = None
145     follow_events_subset = subset[subset['用户行为 (User behaviour)'] == FOLLOW_ACTION_CODE]
146     if not follow_events_subset.empty:
147         actual_follow_time = follow_events_subset['时间 (Time)'].min()
148         print(f"检测到关注行为, 首次关注时间: {actual_follow_time}")
149
150     for i, t_point in enumerate(output_time_index):
151         t_point_date = t_point.date()
152
153         # 1. 基于 actual_follow_time 检查是否已进入“已关注”状态
154         if not has_followed_final and actual_follow_time is not None and t_point >= actual_follow_time:
155             current_interest_final = FOLLOW_INTEREST_LEVEL

```

```

151     has_followed_final = True
152
153     # 2. 如果已关注, 则兴趣度固定
154     if has_followed_final:
155         current_interest_final = FOLLOW_INTEREST_LEVEL
156     # 3. 如果未关注且兴趣度大于0, 则进行衰减
157     elif current_interest_final > 0 and (t_point - sim_start_time).days >= 5:
158         if last_decay_date_final is not None:
159             days_passed_since_last_decay = (t_point_date - last_decay_date_final).days
160             if days_passed_since_last_decay > 0:
161                 decay_amount = DAILY_DECAY_AMOUNT * days_passed_since_last_decay
162                 current_interest_final = max(0, current_interest_final - decay_amount)
163                 print(f"时间点 {t_point}: 衰减 {decay_amount}, 衰减后兴趣度: {
164                     ↪ current_interest_final}")
165             # 无论是否衰减, 都更新上次衰减检查的日期
166             last_decay_date_final = t_point_date
167
168     # 4. 处理当前时间点 [interval_start, t_point] 内的事件
169     # interval_start应该是上一个output_time_index的时间点, 或者是序列开始前的某个点
170     # t_point是当前output_time_index的时间点
171     # 我们要找的是在 (上一个t_point, 当前t_point] 区间的事件
172     # 对于第一个点, 区间是 (-inf, 第一个t_point]
173     interval_start_event_check = output_time_index[i - 1] if i > 0 else (
174         t_point - pd.to_timedelta(freq)) # 更合理的区间开始
175     # 或者, 对于更精确的事件聚合:
176     # interval_start_event_check = pd.Timestamp.min if i == 0 else output_time_index[i - 1]
177
178     events_in_interval = subset[(subset['时间 (Time)'] > interval_start_event_check) & (subset['时
179         ↪ 间 (Time)'] <= t_point)]
180
181     if not events_in_interval.empty:
182         for _, event in events_in_interval.iterrows():
183             # 再次检查事件是否触发关注状态 (以防事件时间正好在关注时间点或之后)
184             if not has_followed_final and actual_follow_time is not None and event['时间 (Time)']
185                 ↪ >= actual_follow_time:
186                 current_interest_final = FOLLOW_INTEREST_LEVEL
187                 has_followed_final = True
188                 print(f"事件 {event['时间 (Time)']}: 触发关注状态, 兴趣度设为 {
189                     ↪ current_interest_final}")
190
191             if has_followed_final: # 如果在处理事件过程中变为已关注
192                 current_interest_final = FOLLOW_INTEREST_LEVEL
193             elif event['用户行为 (User behaviour)'] == FOLLOW_ACTION_CODE:
194                 current_interest_final = FOLLOW_INTEREST_LEVEL
195                 has_followed_final = True
196                 actual_follow_time = event['时间 (Time)'] # 更新实际关注时间, 以防有多个关注事件
197                 print(f"事件 {event['时间 (Time)']}: 用户行为关注, 兴趣度设为 {
198                     ↪ current_interest_final}")
199             else: # 其他非关注行为, 增加兴趣度
200                 increment = INTEREST_INCREMENTS.get(event['用户行为 (User behaviour)'],
201                     ↪ DEFAULT_INCREMENT)
202                 current_interest_final += total_points*increment / total_points
203                 current_interest_final = max(0, current_interest_final) # 兴趣度不能为负
204                 print(f"事件 {event['时间 (Time)']}: 行为 {event['用户行为 (User behaviour)']}, 增
205                     ↪ 量 {increment}, 当前兴趣度 {current_interest_final}")

```

```

199
200     final_values[i] = current_interest_final
201     # 如果在当前时间点因为事件进入关注状态，则不再需要衰减日期跟踪
202     if not has_followed_final:
203         last_decay_date_final = t_point_date # 更新衰减日期跟踪
204
205     final_series = pd.Series(final_values, index=output_time_index, name="Interest")
206
207     if start_time_requested is not None:
208         final_series = final_series[final_series.index >= start_time_requested]
209     if end_time_requested is not None:
210         final_series = final_series[final_series.index <= end_time_requested]
211
212     print(f"compute_interest_series_new_model: 计算完成。最终序列长度: {len(final_series)}")
213     return final_series
214
215
216 # --- 绘图函数 ---
217 def plot_interest_new_model(
218     user_id_to_plot: str, # 明确ID为字符串类型
219     up_id_to_plot: str, # 明确ID为字符串类型
220     df_all: Optional[pd.DataFrame] = None,
221     start_time_str_input: Optional[str] = None,
222     end_time_str_input: Optional[str] = None,
223     freq_input: str = '1H'
224 ) -> plt.Figure:
225     """
226     加载数据（如果未提供），计算并绘制用户对博主的兴趣度（新模型）。
227     """
228     if df_all is None:
229         print("plot_interest_new_model: df_all 为空，正在加载数据...")
230         df_all = load_data() # 默认路径
231     else:
232         print("plot_interest_new_model: df_all 已提供。")
233
234     parsed_start_time = None
235     if start_time_str_input and start_time_str_input.strip():
236         try:
237             parsed_start_time = pd.to_datetime(start_time_str_input)
238         except Exception as e:
239             print(f"警告：无法解析开始时间 '{start_time_str_input}'，将自动确定开始时间。错误: {e}")
240
241     parsed_end_time = None
242     if end_time_str_input and end_time_str_input.strip():
243         try:
244             parsed_end_time = pd.to_datetime(end_time_str_input)
245         except Exception as e:
246             print(f"警告：无法解析结束时间 '{end_time_str_input}'，将自动确定结束时间。错误: {e}")
247
248     print(f"plot_interest_new_model: 尝试为用户 '{user_id_to_plot}'，博主 '{up_id_to_plot}' 计算兴趣度"
249           ↪ )
250     print(
251         f"plot_interest_new_model: 请求开始时间: {parsed_start_time}, 请求结束时间: {parsed_end_time},
252         ↪ 频率: {freq_input}")

```

```

252     try:
253         interest_series = compute_interest_series_new_model(
254             df_all,
255             user_id_to_plot,
256             up_id_to_plot,
257             start_time_requested=parsed_start_time,
258             end_time_requested=parsed_end_time,
259             freq=freq_input
260         )
261         print(f"plot_interest_new_model: 兴趣度序列计算完成。长度: {len(interest_series)}")
262         if not interest_series.empty:
263             print(f"plot_interest_new_model: 计算得到的兴趣度序列 (前5条数据, 如果存在):\n{
264                 ↪ interest_series.head()}")
265             print(f"plot_interest_new_model: 计算得到的兴趣度序列 (后5条数据, 如果存在):\n{
266                 ↪ interest_series.tail()}")
267         else:
268             print("plot_interest_new_model: 计算得到的兴趣度序列为空。")
269
270     except ValueError as e:
271         print(f"计算兴趣度时出错 (在 plot_interest_new_model 中): {e}")
272         return None
273     except Exception as e:
274         import traceback
275         print(f"计算兴趣度时发生未预料的错误 (在 plot_interest_new_model 中): {e}")
276         traceback.print_exc()
277         return None
278
279 fig = plt.figure(figsize=(12, 6))
280 if interest_series.empty:
281     print(f"没有可为用户 '{user_id_to_plot}' 和博主 '{up_id_to_plot}' 绘制的兴趣数据。(序列为空)")
282     plt.title(f"用户 '{user_id_to_plot}' 对博主 '{up_id_to_plot}' 的兴趣度 (无数据)", fontsize=14)
283     plt.xlabel('时间', fontsize=12)
284     plt.ylabel('兴趣度水平', fontsize=12)
285     plt.text(0.5, 0.5, '无交互数据', horizontalalignment='center', verticalalignment='center',
286             transform=plt.gca().transAxes)
287     plt.grid(True, linestyle='--', alpha=0.7)
288     plt.tight_layout()
289 else:
290     print("plot_interest_new_model: 准备绘图...")
291     plt.step(interest_series.index, interest_series.values, where='post', marker='.', linestyle='-'
292             ↪ )
293     plt.title(f"用户 '{user_id_to_plot}' 对博主 '{up_id_to_plot}' 的兴趣度 (状态模型)", fontsize
294             ↪ =14)
295     plt.xlabel('时间', fontsize=12)
296     plt.ylabel('兴趣度水平', fontsize=12)
297     plt.xticks(rotation=45)
298     plt.grid(True, linestyle='--', alpha=0.7)
299     plt.tight_layout()
300
301 return fig
302
303 # --- 主程序执行部分 ---
304 if __name__ == '__main__':
305     c1_file_path = 'C1.csv'

```

```

303 c2_file_path = 'C2.csv'
304 default_freq = '30min'
305
306 print("--- 用户兴趣度分析程序（状态模型）---")
307
308 # 获取用户输入，并直接作为字符串处理
309 user_id_input_str = input("请输入用户ID（例如 U1, U2642）: ").strip()
310 up_id_input_str = input("请输入博主ID（用“、”分隔多个ID，例如 B60、B123）: ").strip()
311 up_ids = up_id_input_str.split(',')
312
313 start_time_input = input(
314     f"请输入开始时间（格式 YYYY-MM-DD HH:MM 或 YYYY-MM-DD，直接回车则根据数据自动确定）: ").strip()
315 end_time_input = input(
316     f"请输入结束时间（格式 YYYY-MM-DD HH:MM 或 YYYY-MM-DD，直接回车则根据数据自动确定）: ").strip()
317 freq_input_val = input(
318     f"请输入时间频率（例如 '1H', '1D', '30min', 直接回车使用默认 '{default_freq}'）: ").strip() or
    ↪ default_freq
319
320 try:
321     print(f"准备从 {c1_file_path} 和 {c2_file_path} 加载数据...")
322     # 在调用load_data时可以传入文件路径
323     df_loaded = load_data(c1_path=c1_file_path, c2_path=c2_file_path)
324
325     # 打印加载后DataFrame的列信息和数据类型，有助于调试ID匹配问题
326     print("--- 加载的DataFrame信息 ---")
327     print("列名:", df_loaded.columns.tolist())
328     print("数据类型:\n", df_loaded.dtypes)
329     # print("前5条数据:\n", df_loaded.head())
330     print("-----")
331
332     # 创建PDF文件
333     pdf_path = f'interest_plots_{user_id_input_str}.pdf'
334     c = canvas.Canvas(pdf_path, pagesize=letter)
335
336     for up_id in up_ids:
337         print(
338             f"准备为用户: '{user_id_input_str}'（类型: {type(user_id_input_str)}），博主: '{up_id}'
            ↪ （类型: {type(up_id)}）绘制兴趣度图")
339         fig = plot_interest_new_model(
340             user_id_input_str,
341             up_id,
342             df_all=df_loaded,
343             start_time_str_input=start_time_input if start_time_input else None,
344             end_time_str_input=end_time_input if end_time_input else None,
345             freq_input=freq_input_val
346         )
347
348         if fig:
349             # 保存图像到临时文件
350             temp_img_path = f'temp_plot_{up_id}.png'
351             fig.savefig(temp_img_path)
352             plt.close(fig)
353
354             # 将图像添加到PDF
355             c.drawImage(temp_img_path, 50, 100, width=500, height=300)

```



```

356         c.showPage()
357
358     # 保存PDF文件
359     c.save()
360     print(f"兴趣度图像已保存到 {pdf_path}")
361
362     except FileNotFoundError:
363         print("退出程序：一个或多个数据文件未找到。请检查文件名和路径。")
364     except ValueError as ve:
365         print(f"发生值错误（可能是数据格式、列名问题或ID类型不匹配）：{ve}")
366         import traceback
367         traceback.print_exc()
368     except Exception as e:
369         import traceback
370         print(f"发生未预料的错误：{e}")
371         traceback.print_exc()
372
373     print("--- 程序执行完毕 ---")

```

10.2 绘制 Fourier 函数图像

```

1 import re
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from matplotlib.backends.backend_pdf import PdfPages
5 plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
6
7 def parse_fourier_series(equation_str):
8     """
9     Parses a Fourier series string to extract DC component, amplitudes,
10     frequency multipliers, and phase shifts.
11     """
12     # Remove "f(t) =" prefix if present
13     equation_str = re.sub(r'^f\(t\)s*=\s*', '', equation_str).strip()
14
15     # Extract DC component (real part of a potentially complex number like "14.2639-0.0000j")
16     dc_match = re.match(r'^(-?\d+\.\d+)(?:[+-]\d+\.\d+j)?', equation_str)
17     if not dc_match:
18         dc_match_simple = re.match(r'^(-?\d+\.\d+)', equation_str)
19         if not dc_match_simple:
20             raise ValueError(f"Could not find DC component in the equation start: '{equation_str[:50]}...'")
21         dc_component = float(dc_match_simple.group(1))
22         equation_str = equation_str[len(dc_match_simple.group(0)):].strip()
23     else:
24         dc_component = float(dc_match.group(1)) # Group 1 is the real part
25         equation_str = equation_str[len(dc_match.group(0)):].strip()
26
27     # Extract cosine terms: Amplitude * cos(2 * N*t/T + Phase)
28     terms = re.findall(r'(\d+\.\d+)\s*\*\s*cos\((2 \*(\d+)\s*\s*t/(\d+)\s*\s*\s*\s*(-?\d+\.\d+)\s*)\)', equation_str)
29
30     amplitudes = []
31     freq_multipliers = []
32     phases = []

```

```

33     period_val = None
34
35     if not terms and equation_str.strip() and not equation_str.strip().startswith('+'):
36         print(f"Warning: No cosine terms found or regex mismatch after DC component. Remaining string:
37             ↳ '{equation_str[:100]}...'")
38
39     for term_match in terms:
40         amplitudes.append(float(term_match[0]))
41         freq_multipliers.append(int(term_match[1]))
42         current_term_period = int(term_match[2])
43         if period_val is None:
44             period_val = current_term_period
45         elif period_val != current_term_period:
46             raise ValueError(f"Inconsistent period T found in terms. Expected {period_val}, got {
47                 ↳ current_term_period}.")
48         phases.append(float(term_match[3]))
49
50     if not period_val and amplitudes:
51         raise ValueError("Period T could not be determined from cosine terms.")
52     elif not amplitudes:
53         if period_val is None:
54             period_val = 72
55             # print(f"No cosine terms to derive period, defaulting to {period_val} for calculations if
56                 ↳ needed.")
57
58     return dc_component, amplitudes, freq_multipliers, phases, period_val if period_val is not None
59         ↳ else 72
60
61 def fourier_series_function_calc(t, dc, amps, freqs, phases, period_calc):
62     """
63     Calculates the value of the Fourier series at time t.
64     """
65     y = dc
66     for i in range(len(amps)):
67         y += amps[i] * np.cos(2 * np.pi * freqs[i] * t / period_calc + phases[i])
68     return y
69
70 # Specify the correct encoding for the file
71 file_path = 'fourier_formulas.txt'
72 file_encoding = 'gbk' # Changed from 'utf-8' to 'gbk'
73
74 try:
75     with open(file_path, 'r', encoding=file_encoding) as f:
76         content = f.read()
77 except FileNotFoundError:
78     print(f"错误: 文件 {file_path} 未找到。")
79     exit()
80 except UnicodeDecodeError:
81     print(f"错误: 使用 {file_encoding} 编码无法解码文件。请尝试其他编码, 例如 'gb2312'。")
82     exit()
83 except Exception as e:
84     print(f"读取文件时出错: {e}")
85     exit()
86
87 series_strings_from_file = []

```

```

84 # The file content has "列 1:" and "列 2:" prefixes, and terms can be split by newlines.
85 # Split the content by "列 " to separate the series definitions.
86 # Then re-add "列 " for consistency if needed by parsing, or just take the part after ":".
87 raw_series_parts = content.split("列 ")
88 # raw_series_parts will have an empty string at the beginning if "列 " is at the start of the file.
89
90 for part in raw_series_parts:
91     if not part.strip():
92         continue
93
94     # Remove potential numbering like "1:", "2:" and "f(t) =" before actual formula
95     # Each part might look like "1: f(t) = 14.2639-0.0000j + ..."
96     # Or "2: f(t) = 63.3472-0.0000j + ..."
97     # We need to ensure line breaks within a formula are handled by joining them.
98     actual_formula_lines = part.split(":", 1)
99     if len(actual_formula_lines) > 1:
100         # Join lines that might have been part of the formula itself (e.g. if source had newlines in
101             ↳ formula)
102         formula_str = actual_formula_lines[1].replace('\n', ' ').replace('\r', ' ').strip()
103         series_strings_from_file.append(formula_str)
104
105 if not series_strings_from_file or all(not s for s in series_strings_from_file):
106     print(f"错误: 文件中未找到有效的傅里叶级数数据。")
107     exit()
108
109 # Define time range
110 t_plot_values = np.linspace(0, 72, 2000, endpoint=False) # From 0 up to (but not including) 72
111
112 output_pdf_file_path = 'fourier_series_plots_corrected.pdf'
113 try:
114     with PdfPages(output_pdf_file_path) as pdf:
115         for i, series_data_str in enumerate(series_strings_from_file):
116             if not series_data_str.strip():
117                 print(f"跳过索引 {i} 处的空级数数据。")
118                 continue
119
120             plt.figure(figsize=(12, 7))
121             try:
122                 print(f"正在处理级数 {i+1}...")
123
124                 dc_val, amplitudes_val, freqs_val, phases_val, period_val_parsed = parse_fourier_series
125                     ↳ (series_data_str)
126
127                 y_plot_values = fourier_series_function_calc(t_plot_values, dc_val, amplitudes_val,
128                     ↳ freqs_val, phases_val, period_val_parsed)
129
130                 plt.plot(t_plot_values, y_plot_values, label=f'级数 {i+1} 数据')
131                 plt.title(f'傅里叶级数 {i+1}', fontsize=16)
132                 plt.xlabel('t', fontsize=14)
133                 plt.ylabel(f'f{i+1}(t)', fontsize=14)
134                 plt.grid(True, linestyle='--', alpha=0.7)
135
136                 # Matplotlib may need specific font support for Chinese characters in legend/text
137                 # This is a general plot, for specific character display, font setup might be needed.

```

```

136 plt.legend(prop={'family': 'SimHei'}) # Example: Use SimHei font if available for
    ↪ Chinese
137
138 plt.margins(x=0.01, y=0.05)
139 plt.axhline(0, color='black', linewidth=0.5)
140 plt.axvline(0, color='black', linewidth=0.5)
141
142 info_text = f"直流分量 (DC): {dc_val:.4f}\n"
143 info_text += f"谐波数量: {len(amplitudes_val)}\n"
144 if amplitudes_val:
145     info_text += f"第1谐波: 幅值={amplitudes_val[0]:.2f}, 频率倍数={freqs_val[0]}, 相位
    ↪ ={phases_val[0]:.2f} rad\n"
146     if len(amplitudes_val) > 1:
147         info_text += f"最后谐波 ({len(amplitudes_val)}): 幅值={amplitudes_val[-1]:.2f}
    ↪ }, 频率倍数={freqs_val[-1]}, 相位={phases_val[-1]:.2f} rad"
148
149 # Add text annotation; ensure font can display Chinese if needed
150 plt.text(0.02, 0.98, info_text, transform=plt.gca().transAxes, fontsize=9,
151         verticalalignment='top', bbox=dict(boxstyle='round,pad=0.5', fc='aliceblue',
    ↪ alpha=0.8),
152         family='SimHei') # Example font
153
154 plt.tight_layout(rect=[0, 0, 1, 0.96])
155 pdf.savefig(bbox_inches='tight')
156 plt.close()
157 print(f"级数 {i+1} 绘制成功。")
158
159 except ValueError as ve_parse:
160     print(f"解析或处理级数 {i+1} 时出错: {ve_parse}")
161     plt.text(0.5, 0.5, f"处理级数 {i+1} 时出错:\n{ve_parse}\n\n问题字符串部分:\n{
    ↪ series_data_str[:150]}...",
162             horizontalalignment='center', verticalalignment='center',
163             fontsize=10, color='red', wrap=True, family='monospace')
164     plt.title(f"傅里叶级数 {i+1} - 解析错误", fontsize=14)
165     plt.gca().set_axis_off()
166     pdf.savefig(bbox_inches='tight')
167     plt.close()
168 except Exception as e_general:
169     print(f"处理级数 {i+1} 时发生意外错误: {e_general}")
170     plt.text(0.5, 0.5, f"级数 {i+1} 发生意外错误:\n{type(e_general).__name__}: {e_general}"
    ↪ ,
171             horizontalalignment='center', verticalalignment='center',
172             fontsize=10, color='red', wrap=True, family='monospace')
173     plt.title(f"傅里叶级数 {i+1} - 意外错误", fontsize=14)
174     plt.gca().set_axis_off()
175     pdf.savefig(bbox_inches='tight')
176     plt.close()
177
178 print(f"成功生成PDF: {output_pdf_file_path}")
179
180 except Exception as e_pdf:
181     print(f"无法创建PDF: {e_pdf}")
182     output_pdf_file_path = None

```

10.3 绘制各博主逐小时互动频次表

```

1 import pandas as pd
2
3 # 加载数据
4 df = pd.read_csv('C1.csv')
5
6 print('数据基本信息: ')
7 df.info()
8
9 # 查看数据集行数和列数
10 rows, columns = df.shape
11
12 import matplotlib.pyplot as plt
13
14 # 将时间列转换为日期时间格式
15 df['时间 (Time)'] = pd.to_datetime(df['时间 (Time)'])
16
17 # 筛选用户行为为4的数据
18 filtered_df = df[df['用户行为 (User behaviour)'] == 4]
19
20 # 按小时和博主ID分组, 统计用户行为为4的次数
21 hourly_counts = filtered_df.groupby([filtered_df['时间 (Time)'].dt.strftime('%Y-%m-%d %H:00'), '博主ID
    ↳ (Blogger ID)'])['用户行为 (User behaviour)'].count().unstack(fill_value=0)
22
23 output_file = '逐小时评论数表格.xlsx'
24 hourly_counts.to_excel(output_file, sheet_name='用户行为统计')
25
26 print(f'数据已成功保存到 {output_file}')

```

10.4 检索代码

```

1 import pandas as pd
2
3 # 定义要查找的用户ID
4 target_user_id = 'U26447'
5 input_files = ['C1.csv', 'C2.csv']
6
7 # 用于存储所有匹配记录的列表
8 all_records = []
9
10 # 遍历处理每个CSV文件
11 for file in input_files:
12     try:
13         # 读取CSV文件
14         df = pd.read_csv(file)
15         # 查找特定用户ID的所有记录
16         user_records = df[df['用户ID (User ID)'] == target_user_id]
17         # 将匹配的记录添加到结果列表
18         all_records.append(user_records)
19         print(f"从 {file} 中找到 {len(user_records)} 条匹配记录")
20     except FileNotFoundError:
21         print(f"错误: 找不到文件 {file}, 请检查文件路径")
22     except Exception as e:
23         print(f"处理文件 {file} 时出错: {e}")

```

```

24
25 # 合并所有匹配的记录
26 if all_records:
27     combined_records = pd.concat(all_records, ignore_index=True)
28     # 保存结果到新CSV
29     output_file = f"{target_user_id}_records.csv"
30     combined_records.to_csv(output_file, index=False)
31     print(f"已将所有匹配记录保存到 {output_file}")
32 else:
33     print("未找到任何匹配的记录")

```

10.5 关注可能性函数

```

1 import pandas as pd
2 import numpy as np
3 from datetime import datetime, timedelta
4
5
6 def load_data():
7     print("开始加载数据...")
8     # C1数据: 7.11-7.20 (所有用户历史数据)
9     try:
10         df_c1 = pd.read_csv('C1数据.csv', parse_dates=['时间'], encoding='utf-8-sig')
11         print("C1数据.csv 加载成功。")
12     except FileNotFoundError:
13         print("错误: C1数据.csv 未找到。此文件对于历史分析至关重要。")
14         raise
15     except Exception as e:
16         print(f"加载 C1数据.csv 时出错: {e}")
17         print(f"请确保 C1数据.csv 文件存在, 并且列名为 ['用户ID', '用户行为', '博主ID', '时间']。")
18         raise
19     try:
20         df_c2 = pd.read_csv('C2.csv', parse_dates=['时间'], encoding='utf-8-sig')
21         print("C2.csv 加载成功。")
22     except FileNotFoundError:
23         print("错误: C2.csv 未找到。")
24         raise
25     except Exception as e:
26         print(f"加载 C2.csv 时出错: {e}")
27         print(f"请确保 C2.csv 文件存在, 并且列名为 ['用户ID', '用户行为', '博主ID', '时间']。")
28         raise
29
30     try:
31         df_u7 = pd.read_excel('U7.xlsx', parse_dates=['时间'])
32         print("U7.xlsx 加载成功。")
33     except FileNotFoundError:
34         df_u7 = pd.DataFrame()
35         print("警告: U7.xlsx 未找到, 将视为空数据。")
36     except Exception as e:
37         print(f"加载 U7.xlsx 时出错: {e}。 请检查文件格式和列名。")
38         df_u7 = pd.DataFrame()
39
40     try:
41         df_u52010 = pd.read_excel('U52010.xlsx', parse_dates=['时间'])
42         print("U52010.xlsx 加载成功。")

```

```

43 except FileNotFoundError:
44     df_u52010 = pd.DataFrame()
45     print("警告: U52010.xlsx 未找到, 将视为空数据。")
46 except Exception as e:
47     print(f"加载 U52010.xlsx 时出错: {e}。 请检查文件格式和列名。")
48     df_u52010 = pd.DataFrame()
49
50 try:
51     df_u5769 = pd.read_excel('U5769.xlsx', parse_dates=['时间'])
52     print("U5769.xlsx 加载成功。")
53 except FileNotFoundError:
54     df_u5769 = pd.DataFrame()
55     print("警告: U5769.xlsx 未找到, 将视为空数据。")
56 except Exception as e:
57     print(f"加载 U5769.xlsx 时出错: {e}。 请检查文件格式和列名。")
58     df_u5769 = pd.DataFrame()
59
60 try:
61     df_u14990 = pd.read_excel('U14990.xlsx', header=None).rename(
62         columns={0: '用户ID', 1: '用户行为', 2: '博主ID', 3: '时间'})
63     df_u14990['时间'] = pd.to_datetime(df_u14990['时间'])
64     print("U14990.xlsx 加载成功。")
65 except FileNotFoundError:
66     df_u14990 = pd.DataFrame()
67     print("警告: U14990.xlsx 未找到, 将视为空数据。")
68 except Exception as e:
69     print(f"加载 U14990.xlsx 时出错: {e}。 请检查文件格式和列名。")
70     df_u14990 = pd.DataFrame()
71
72 try:
73     df_u6749 = pd.read_excel('U6749.xlsx', header=None).rename(
74         columns={0: '用户ID', 1: '用户行为', 2: '博主ID', 3: '时间'})
75     df_u6749['时间'] = pd.to_datetime(df_u6749['时间'])
76     print("U6749.xlsx 加载成功。")
77 except FileNotFoundError:
78     df_u6749 = pd.DataFrame()
79     print("警告: U6749.xlsx 未找到, 将视为空数据。")
80 except Exception as e:
81     print(f"加载 U6749.xlsx 时出错: {e}。 请检查文件格式和列名。")
82     df_u6749 = pd.DataFrame()
83
84 all_dfs = [df_c1, df_c2, df_u7, df_u52010, df_u5769, df_u14990, df_u6749]
85 valid_dfs = [df for df in all_dfs if
86     not df.empty and all(col in df.columns for col in ['用户ID', '用户行为', '博主ID', '时
87         ↪ 间'])]
88
89 if not valid_dfs:
90     print("错误: 没有成功加载任何包含必需列的数据文件。程序无法继续。")
91     print("必需列为: ['用户ID', '用户行为', '博主ID', '时间']")
92     raise ValueError("无有效数据加载或必需列缺失")
93
94 df_combined = pd.concat(valid_dfs, ignore_index=True)
95 print(f"数据合并完成。总记录数: {len(df_combined)}")
96
97 required_cols_for_ops = ['用户ID', '用户行为', '博主ID', '时间']

```

```

97     if not all(col in df_combined.columns for col in required_cols_for_ops):
98         missing_cols = [col for col in required_cols_for_ops if col not in df_combined.columns]
99         print(f"错误: 合并后的DataFrame缺少以下必需列: {missing_cols}")
100         raise ValueError(f"合并后的DataFrame缺少必需列: {missing_cols}")
101
102     df_combined['用户ID'] = df_combined['用户ID'].astype(str)
103     df_combined['博主ID'] = df_combined['博主ID'].astype(str)
104     df_combined['用户行为'] = pd.to_numeric(df_combined['用户行为'], errors='coerce').fillna(0).astype(
        ↪ int)
105
106     df_combined.dropna(subset=['时间'], inplace=True)
107     df_combined.sort_values(by='时间', inplace=True)
108
109     print("数据预处理完成。")
110     return df_combined
111
112
113 def interest_calculate(df: pd.DataFrame, user_id: str, up_id: str, beginning: datetime, ending:
        ↪ datetime) -> float:
114     mask = (df['用户ID'] == user_id) & \
115            (df['博主ID'] == up_id) & \
116            (df['时间'] >= beginning) & \
117            (df['时间'] < ending) & \
118            (df['用户行为'] != 4)
119     interactions = df[mask]
120
121     interest = 0.0
122     for _, row in interactions.iterrows():
123         if row['用户行为'] == 1:
124             interest += 0.5
125         elif row['用户行为'] == 2:
126             interest += 2.0
127         elif row['用户行为'] == 3:
128             interest += 3.0
129     return interest
130
131
132 def deadline_interest(df: pd.DataFrame, user_id: str) -> float:
133     follow_records = df[(df['用户ID'] == user_id) & (df['用户行为'] == 4)].copy()
134
135     if follow_records.empty:
136         return 0.0
137
138     pre_follow_interests = []
139     if df.empty or df['时间'].isna().all():
140         min_global_time = datetime.min
141     else:
142         min_global_time = df['时间'].min()
143
144     for _, follow_event in follow_records.iterrows():
145         up_id_followed = follow_event['博主ID']
146         time_of_follow = follow_event['时间']
147         interest_before_this_follow = interest_calculate(df, user_id, up_id_followed, min_global_time,
            ↪ time_of_follow)
148         if interest_before_this_follow > 0: # 只考虑有实际兴趣积累的情况

```



```

149         pre_follow_interests.append(interest_before_this_follow)
150
151     if not pre_follow_interests:
152         return 0.0
153     # 原来是 min(pre_follow_interests) * 0.85
154     # 修改为 min(pre_follow_interests) * 0.60 (或者您可以尝试其他值, 如0.5, 0.7)
155     threshold_value = min(pre_follow_interests) * 0.60
156     # print(f" [Debug] User {user_id}: pre_follow_interests={pre_follow_interests}, min_interest={min(
157         ↪ pre_follow_interests)}, threshold_value={threshold_value:.2f}")
158     return threshold_value
159
160 def seeking_proportion_n_1(df: pd.DataFrame, user_id: str, up_id: str) -> int:
161     date_22 = datetime(2024, 7, 22)
162     date_20 = datetime(2024, 7, 20)
163     date_19 = datetime(2024, 7, 19)
164     date_18 = datetime(2024, 7, 18)
165     date_15 = datetime(2024, 7, 15)
166
167     A1 = interest_calculate(df, user_id, up_id, date_22, date_22 + timedelta(days=1))
168
169     interests_for_A2 = [
170         interest_calculate(df, user_id, up_id, date_18, date_18 + timedelta(days=1)),
171         interest_calculate(df, user_id, up_id, date_19, date_19 + timedelta(days=1)),
172         interest_calculate(df, user_id, up_id, date_20, date_20 + timedelta(days=1))
173     ]
174     # 处理A2可能为空列表的情况 (如果那几天没有任何互动)
175     A2 = np.mean([i for i in interests_for_A2 if pd.notnull(i) and i > 0]) if any(
176         pd.notnull(i) and i > 0 for i in interests_for_A2) else 0.0
177
178     interests_for_A3 = [
179         interest_calculate(df, user_id, up_id, date_19, date_19 + timedelta(days=1)),
180         interest_calculate(df, user_id, up_id, date_20, date_20 + timedelta(days=1)),
181         A1
182     ]
183     A3 = np.mean([i for i in interests_for_A3 if pd.notnull(i) and i > 0]) if any(
184         pd.notnull(i) and i > 0 for i in interests_for_A3) else 0.0
185
186     interests_for_A4 = []
187     current_date = date_15
188     while current_date <= date_20:
189         interests_for_A4.append(interest_calculate(df, user_id, up_id, current_date, current_date +
190             ↪ timedelta(days=1)))
191         current_date += timedelta(days=1)
192     A4 = np.mean([i for i in interests_for_A4 if pd.notnull(i) and i > 0]) if any(
193         pd.notnull(i) and i > 0 for i in interests_for_A4) else 0.0
194
195     condition_met = False
196     if A1 > 0:
197         if A2 == 0 and A4 == 0:
198             condition_met = True
199         elif A1 > A2 and (A3 > A2 or A2 > A4 or A3 > A4):
200             condition_met = True
201         elif A1 > A2 and A2 == 0 and A4 > 0:
202             condition_met = True

```

```

202
203     if condition_met:
204         return 1
205     else:
206         return 0
207
208
209 def judge_first(df: pd.DataFrame, user_id: str, up_id: str) -> int:
210     user_threshold = deadline_interest(df, user_id)
211     date_22 = datetime(2024, 7, 22)
212     interest_on_july22 = interest_calculate(df, user_id, up_id, date_22, date_22 + timedelta(days=1))
213
214     if interest_on_july22 > 0 and interest_on_july22 >= user_threshold:
215
216         return 1
217     else:
218         return 0
219
220
221 def judge_result(df: pd.DataFrame, user_id: str, up_id: str) -> int:
222     trend_positive = seeking_proportion_n_1(df, user_id, up_id)
223     threshold_met = judge_first(df, user_id, up_id)
224     if trend_positive == 1 and threshold_met == 1:
225         return 1
226     else:
227         return 0
228
229
230 def predict_follows_for_users(df: pd.DataFrame, target_user_ids: list) -> dict:
231     predictions = {}
232     date_22_obj = datetime(2024, 7, 22).date()
233
234     for user_id in target_user_ids:
235         print(f"\n处理用户: {user_id}")
236         predicted_up_ids_for_user = []
237
238         user_interactions_on_july22 = df[
239             (df['用户ID'] == user_id) &
240             (df['时间'].dt.date == date_22_obj)
241         ]
242
243         if user_interactions_on_july22.empty:
244             print(f" 用户 {user_id} 在7月22日没有互动记录。")
245             predictions[user_id] = []
246             continue
247
248         up_ids_interacted_with = user_interactions_on_july22['博主ID'].unique()
249         print(f" 用户 {user_id} 在7月22日与以下UP主有互动: {list(up_ids_interacted_with)}")
250         for up_id in up_ids_interacted_with:
251             already_followed_mask = (df['用户ID'] == user_id) & \
252                                     (df['博主ID'] == up_id) & \
253                                     (df['用户行为'] == 4) & \
254                                     (df['时间'].dt.date <= date_22_obj)
255             if not df[already_followed_mask].empty:
256                 print(f" >> 信息: 用户 {user_id} 已关注UP主 {up_id} (或在7月22日关注), 跳过预测。")

```

```

257         continue
258
259     print(f" 正在分析用户 {user_id} 与UP主 {up_id} (尚未关注) 的关系...")
260     if judge_result(df, user_id, up_id) == 1:
261         predicted_up_ids_for_user.append(up_id)
262         print(f"  >> 预测用户 {user_id} 将关注UP主 {up_id}")
263
264     predictions[user_id] = predicted_up_ids_for_user
265
266     return predictions
267
268
269 if __name__ == "__main__":
270     try:
271         df_main = load_data()
272
273         if df_main is None or df_main.empty:
274             print("数据加载失败或数据为空，无法进行预测。")
275         else:
276             users_to_predict = ['U7', 'U52010', 'U5769', 'U14990', 'U6749']
277             actual_users_in_df = df_main['用户ID'].unique()
278             users_to_process = [u for u in users_to_predict if u in actual_users_in_df]
279             missing_users = [u for u in users_to_predict if u not in actual_users_in_df]
280
281             if missing_users:
282                 print(f"警告：以下指定的用户ID在加载的数据中未找到，将不会为他们进行预测：{
283                     ↪ missing_users}")
284             if not users_to_process:
285                 print("错误：所有指定的用户ID均未在加载数据中找到。无法进行预测。")
286             else:
287                 final_predictions = predict_follows_for_users(df_main, users_to_process)
288
289                 print("\n--- 最终预测结果 ---")
290                 total_predicted_follows = 0
291                 for user, followed_ups in final_predictions.items():
292                     if followed_ups:
293                         print(f"用户 {user} 预计将在7月22日关注的UP主：{'， '.join(followed_ups)}")
294                         total_predicted_follows += len(followed_ups)
295                     else:
296                         print(f"用户 {user} 预计在7月22日不会新增关注（基于当前分析的UP主）。")
297                 for missing_user in missing_users:
298                     print(f"用户 {missing_user} 未在数据中找到，无预测结果。")
299
300                 print(f"\n总共预测到 {total_predicted_follows} 条新增关注。")
301
302     except FileNotFoundError as fnf_error:
303         print(f"程序终止：必要文件未找到。{fnf_error}")
304     except ValueError as val_error:
305         print(f"程序终止：数据问题。{val_error}")
306     except Exception as e:
307         print(f"程序执行过程中发生未预料的错误：{e}")
308         import traceback
309
310         traceback.print_exc()

```

10.6 上线检测

```

1 import pandas as pd
2 import numpy as np
3 from datetime import datetime, timedelta
4
5
6 # --- 数据加载函数（直接加载用户特定的xlsx文件） ---
7 def load_user_specific_data(user_files_info: dict) -> pd.DataFrame:
8     print("开始加载用户特定数据...")
9     all_user_dfs = []
10    for user_id_key, filename in user_files_info.items():
11        try:
12            # 假设xlsx文件的列名是纯中文：'用户ID', '用户行为', '博主ID', '时间'
13            df_user = pd.read_excel(filename, parse_dates=['时间'])
14
15            if '用户ID' not in df_user.columns:
16                print(f"警告：文件 {filename} 中未找到 '用户ID' 列。将使用 '{user_id_key}' 作为用户ID。
17                    ↪ ")
18                df_user['用户ID'] = user_id_key
19            else:
20                df_user['用户ID'] = df_user['用户ID'].astype(str)
21                # if not all(df_user['用户ID'] == user_id_key): # 可选的检查
22                #     print(f"警告：文件 {filename} 中的 '用户ID' 与文件名key '{user_id_key}' 不完全匹
23                    ↪ 配。请检查数据。")
24
25            # 确保其他必要列存在或为其创建默认值
26            # 核心列是 '时间' 和 '用户ID', 其他列如果缺失, 模型也能运行 (只是部分特征可能无意义)
27            base_required_cols = ['时间', '用户ID']
28            optional_cols = ['用户行为', '博主ID']
29
30            for col in base_required_cols:
31                if col not in df_user.columns:
32                    print(f"错误：文件 {filename} 缺少核心列 '{col}'。跳过此文件。")
33                    raise ValueError(f"文件 {filename} 缺少核心列 '{col}'") # 或者 continue
34
35            for col in optional_cols:
36                if col not in df_user.columns:
37                    print(f"警告：文件 {filename} 缺少可选列 '{col}'。将使用默认值或忽略。")
38                    if col == '用户行为':
39                        df_user[col] = 0 # 默认为无特定行为
40                    elif col == '博主ID':
41                        df_user[col] = "UNKNOWN_BLOGGER" # 默认博主ID
42
43            all_user_dfs.append(df_user)
44            print(f"{filename} 加载成功。")
45        except FileNotFoundError:
46            print(f"警告：{filename} 未找到。")
47        except Exception as e:
48            print(f"加载 {filename} 时出错：{e}")
49
50    if not all_user_dfs:
51        print("错误：没有成功加载任何用户数据文件。程序无法继续。")
52        raise ValueError("无有效用户数据加载")

```

```

52 df_combined = pd.concat(all_user_dfs, ignore_index=True)
53 print(f"所有用户数据合并完成。总记录数: {len(df_combined)}")
54
55 df_combined['用户ID'] = df_combined['用户ID'].astype(str)
56 df_combined.dropna(subset=['时间', '用户ID'], inplace=True)
57 df_combined['日期'] = pd.to_datetime(df_combined['时间']).dt.normalize()
58
59 if '用户行为' in df_combined.columns: # 确保列存在再转换
60     df_combined['用户行为'] = pd.to_numeric(df_combined['用户行为'], errors='coerce').fillna(0).
        ↪ astype(int)
61 if '博主ID' in df_combined.columns: # 确保列存在再转换
62     df_combined['博主ID'] = df_combined['博主ID'].astype(str)
63
64 df_combined.sort_values(by='时间', inplace=True)
65 print("数据预处理完成。")
66 return df_combined
67
68
69 # --- 特征工程函数 ---
70 def get_user_login_dates(df_combined: pd.DataFrame, user_id: str) -> pd.Series:
71     """获取指定用户的所有唯一登录日期，并排序"""
72     user_data = df_combined[df_combined['用户ID'] == user_id]
73     if user_data.empty:
74         return pd.Series(dtype='datetime64[ns]')
75     return user_data['日期'].drop_duplicates().sort_values()
76
77
78 def days_since_last_login(user_login_dates: pd.Series, target_date: datetime) -> int:
79     """计算目标日期距离用户最后一次登录的天数"""
80     if user_login_dates.empty:
81         return 999
82     past_logins = user_login_dates[user_login_dates < target_date]
83     if past_logins.empty:
84         return 999
85     last_login_date = past_logins.max()
86     return (target_date - last_login_date).days
87
88
89 def logins_in_last_n_days(user_login_dates: pd.Series, target_date: datetime, n_days: int) -> int:
90     """计算用户在目标日期前n天内的登录天数（不包括目标日期当天）"""
91     if user_login_dates.empty:
92         return 0
93     start_period = target_date - timedelta(days=n_days)
94     recent_logins = user_login_dates[(user_login_dates >= start_period) & (user_login_dates <
        ↪ target_date)]
95     return len(recent_logins)
96
97
98 def logins_on_same_weekday_last_m_weeks(user_login_dates: pd.Series, target_date: datetime, m_weeks:
    ↪ int) -> int:
99     """计算用户在目标日期前m周的同一星期几的登录次数"""
100     if user_login_dates.empty or m_weeks <= 0:
101         return 0
102     target_weekday = target_date.weekday()
103

```

```

104 past_logins = user_login_dates[user_login_dates < target_date]
105 if past_logins.empty:
106     return 0
107
108 start_period_for_weekday_check = target_date - timedelta(weeks=m_weeks)
109 relevant_weekday_logins = past_logins[
110     (past_logins.dt.weekday == target_weekday) &
111     (past_logins >= start_period_for_weekday_check)
112 ]
113 return len(relevant_weekday_logins)
114
115
116 # --- 预测函数（更严格的规则）---
117 def predict_online_status(user_id: str, all_user_login_dates: pd.Series, target_date_dt: datetime) ->
118     ↪ bool:
119     print(f"\n--- 预测用户 {user_id} 在 {target_date_dt.strftime('%Y-%m-%d')} 的上线状态 ---")
120
121     if all_user_login_dates.empty:
122         print(" 用户无历史登录数据，预测为：不上线")
123         return False
124
125     # 特征计算
126     last_seen_days_ago = days_since_last_login(all_user_login_dates, target_date_dt)
127     logins_last_3_days = logins_in_last_n_days(all_user_login_dates, target_date_dt, 3)
128     logins_last_7_days = logins_in_last_n_days(all_user_login_dates, target_date_dt, 7)
129     logins_last_14_days = logins_in_last_n_days(all_user_login_dates, target_date_dt, 14) # 新增特征
130     logins_same_weekday_last_4_weeks = logins_on_same_weekday_last_m_weeks(all_user_login_dates,
131         ↪ target_date_dt, 4)
132
133     print(f" 最后一次登录距今: {last_seen_days_ago} 天")
134     print(f" 过去3天内登录天数: {logins_last_3_days}")
135     print(f" 过去7天内登录天数: {logins_last_7_days}")
136     print(f" 过去14天内登录天数: {logins_last_14_days}")
137     print(f" 过去4周同一星期几登录次数: {logins_same_weekday_last_4_weeks}")
138
139     # --- 判定规则（更严格）---
140
141     # 规则A: 极强活跃信号（近期高频且持续）
142     if logins_last_3_days == 3: # 过去3天每天都上线
143         print(" 判定依据 (A): 过去3天每日上线。预测为：上线")
144         return True
145
146     if logins_last_7_days >= 5 and last_seen_days_ago <= 1: # 过去7天登录至少5天，且昨天或前天刚上过
147         print(" 判定依据 (A): 过去7天高频活跃且非常近期。预测为：上线")
148         return True
149
150     # 规则B: 强不活跃信号（优先排除）
151     if last_seen_days_ago > 10: # 如果超过10天未登录
152         print(f" 判定依据 (B): 已超过10天未登录。预测为：不上线")
153         return False
154
155     if last_seen_days_ago > 5 and logins_last_7_days < 2: # 超过5天未登录，并且过去7天登录不足2次
156         print(f" 判定依据 (B): 超过5天未登录且过去7天活跃度低。预测为：不上线")
157         return False
158
159     if logins_last_14_days < 2 and last_seen_days_ago > 3: # 过去两周总共登录少于2次，且3天前就没上了
160         print(f" 判定依据 (B): 过去14天活跃度极低且近期末登录。预测为：不上线")
161         return False

```

```

157
158 # 规则C: 结合周期性和近期活跃度的较强信号
159 target_weekday_is_tuesday = target_date_dt.weekday() == 1 # 7月23日是星期二
160 if target_weekday_is_tuesday:
161     if logins_same_weekday_last_4_weeks >= 2 and logins_last_7_days >= 2 and last_seen_days_ago <=
        ↪ 3:
162         print(" 判定依据 (C): 历史同星期二活跃, 近期整体活跃且非常近期。预测为: 上线")
163         return True
164
165 # 规则D: 普通活跃信号
166 if logins_last_7_days >= 3 and last_seen_days_ago <= 3:
167     print(f" 判定依据 (D): 过去7天登录至少3次且最近3天内有活动。预测为: 上线")
168     return True
169 if logins_last_3_days >= 1 and logins_last_7_days >= 2 and last_seen_days_ago <= 2:
170     print(f" 判定依据 (D): 近期有多次活动且非常近期。预测为: 上线")
171     return True
172
173 # 规则E: 基于最后登录时间和周期性的较弱信号 (更严格)
174 if last_seen_days_ago <= 3:
175     if target_weekday_is_tuesday and logins_same_weekday_last_4_weeks >= 1:
176         print(f" 判定依据 (E): 最近3天内有活动且历史同星期二有活动。预测为: 上线")
177         return True
178     if logins_last_7_days >= 2:
179         print(f" 判定依据 (E): 最近3天内有活动且过去7天活跃度尚可。预测为: 上线")
180         return True
181
182 # 规则F: 默认不上线
183 print(" 判定依据 (F): 未触发足够的上线信号, 或触发了不活跃信号。预测为: 不上线")
184 return False
185
186
187 # --- 主执行部分 ---
188 if __name__ == "__main__":
189     # 定义用户及其对应的文件名
190     user_files_to_load = {
191         'U9': 'U9.xlsx',
192         'U22405': 'U22405.xlsx',
193         'U16': 'U16.xlsx',
194         'U48420': 'U48420.xlsx'
195     }
196
197     try:
198         df_master = load_user_specific_data(user_files_to_load)
199
200         if df_master.empty:
201             print("没有加载到任何用户数据, 无法进行预测。")
202         else:
203             # 目标预测日期 (7月23日)
204             target_prediction_date_str = "2024-07-23"
205             target_prediction_date = datetime.strptime(target_prediction_date_str, "%Y-%m-%d")
206
207             print(f"\n===== 开始预测用户在 {target_prediction_date_str} 是否上线 =====")
208
209             predictions = {}
210             users_to_predict_online = list(user_files_to_load.keys())

```

```

211
212     for user_id in users_to_predict_online:
213         user_login_history = get_user_login_dates(df_master, user_id)
214
215         if user_id not in df_master['用户ID'].unique():
216             print(f"\n--- 用户 {user_id} 的数据未在合并数据中找到（可能文件加载失败或数据不符）
                ↪ ---")
217             predictions[user_id] = "数据缺失或加载失败"
218             continue
219
220             will_be_online = predict_online_status(user_id, user_login_history,
                ↪ target_prediction_date)
221             predictions[user_id] = "上线" if will_be_online else "不上线"
222
223             print("\n===== 最终预测结果 =====")
224             for user, status in predictions.items():
225                 print(f"用户 {user} 在 {target_prediction_date_str} 预计将: {status}")
226
227     except FileNotFoundError as fnf_error:
228         print(f"程序终止：必要文件未找到。{fnf_error}")
229     except ValueError as val_error:
230         print(f"程序终止：数据问题。{val_error}")
231     except Exception as e:
232         print(f"程序执行过程中发生未预料的错误：{e}")
233         import traceback
234
235         traceback.print_exc()

```

10.7 互动数预测问题三

```

1 import pandas as pd
2 import numpy as np
3 from datetime import datetime, timedelta
4
5
6 # --- load_data 函数（与您上一版本一致，确保能加载所有相关用户数据） ---
7 def load_data():
8     print("开始加载数据...")
9     # 示例框架：确保这里包含了所有需要分析的用户的xlsx文件信息
10    # 以及C1和C2（如果它们包含这些用户的数据）
11    user_files_info = {
12        'U7': 'U7.xlsx', 'U52010': 'U52010.xlsx', 'U5769': 'U5769.xlsx',
13        'U14990': 'U14990.xlsx', 'U6749': 'U6749.xlsx',
14        'U16': 'U16.xlsx', 'U48420': 'U48420.xlsx',
15        'U9': 'U9.xlsx', # 添加 U9
16        'U22405': 'U22405.xlsx' # 添加 U22405
17    }
18    all_dfs = []
19    # 加载C1, C2（如果需要且它们包含U9, U16, U22405, U48420的数据）
20    try:
21        df_c1 = pd.read_csv('C1数据.csv', parse_dates=['时间'], encoding='utf-8-sig')
22        all_dfs.append(df_c1)
23        print("C1数据.csv 加载成功。")
24    except FileNotFoundError:
25        print("警告：C1数据.csv 未找到。")

```



```

26 except Exception as e:
27     print(f"加载 C1数据.csv 时出错: {e}")
28 try:
29     df_c2 = pd.read_csv('C2.csv', parse_dates=['时间'], encoding='utf-8-sig')
30     all_dfs.append(df_c2)
31     print("C2.csv 加载成功。")
32 except FileNotFoundError:
33     print("警告: C2.csv 未找到。")
34 except Exception as e:
35     print(f"加载 C2.csv 时出错: {e}")
36
37 for user_id_key, filename in user_files_info.items():
38     try:
39         df_user = pd.read_excel(filename, parse_dates=['时间'])
40         if '用户ID' not in df_user.columns:
41             df_user['用户ID'] = user_id_key
42         else:
43             df_user['用户ID'] = df_user['用户ID'].astype(str)
44
45         base_required_cols = ['时间', '用户ID']
46         optional_cols = ['用户行为', '博主ID']
47         for col in base_required_cols:
48             if col not in df_user.columns: raise ValueError(f"文件 {filename} 缺少核心列 '{col}'")
49         for col in optional_cols:
50             if col not in df_user.columns:
51                 if col == '用户行为':
52                     df_user[col] = 0
53                 elif col == '博主ID':
54                     df_user[col] = "UNKNOWN_BLOGGER"
55             all_dfs.append(df_user)
56             print(f"{filename} 加载成功。")
57     except FileNotFoundError:
58         print(f"警告: {filename} 未找到。")
59     except Exception as e:
60         print(f"加载 {filename} 时出错: {e}")
61
62 if not all_dfs: raise ValueError("无有效用户数据加载")
63
64 valid_dfs_for_concat = [df for df in all_dfs if not df.empty]
65 if not valid_dfs_for_concat: raise ValueError("所有数据文件均为空或加载失败")
66
67 df_combined = pd.concat(valid_dfs_for_concat, ignore_index=True)
68 print(f"所有用户数据合并完成。总记录数: {len(df_combined)}")
69
70 df_combined['用户ID'] = df_combined['用户ID'].astype(str)
71 df_combined.dropna(subset=['时间', '用户ID'], inplace=True)
72 if '用户行为' in df_combined.columns:
73     df_combined['用户行为'] = pd.to_numeric(df_combined['用户行为'], errors='coerce').fillna(0).
74         ↪ astype(int)
75 else:
76     df_combined['用户行为'] = 0
77 if '博主ID' in df_combined.columns:
78     df_combined['博主ID'] = df_combined['博主ID'].astype(str)
79 else:
80     df_combined['博主ID'] = "UNKNOWN_BLOGGER"

```

```

80     df_combined.sort_values(by='时间', inplace=True)
81     print("数据预处理完成。")
82     return df_combined
83
84
85 # --- 特征工程函数（针对 用户-博主 对） ---
86 def get_interactions_count(df: pd.DataFrame, user_id: str, up_id: str, start_date: datetime, end_date:
87     ↪ datetime) -> int:
88     mask = (df['用户ID'] == user_id) & \
89             (df['博主ID'] == up_id) & \
90             (df['时间'] >= start_date) & \
91             (df['时间'] < end_date)
92     return len(df[mask])
93
94 def get_days_since_last_interaction_with_up(df: pd.DataFrame, user_id: str, up_id: str, target_date:
95     ↪ datetime) -> int:
96     user_up_interactions = df[(df['用户ID'] == user_id) & (df['博主ID'] == up_id) & (df['时间'] <
97     ↪ target_date)]
98     if user_up_interactions.empty: return 999
99     return (target_date - user_up_interactions['时间'].max()).days
100
101 def get_interactions_on_same_weekday_with_up(df: pd.DataFrame, user_id: str, up_id: str, target_date:
102     ↪ datetime,
103     m_weeks: int) -> int:
104     if m_weeks <= 0: return 0
105     target_weekday = target_date.weekday()
106     start_period = target_date - timedelta(weeks=m_weeks)
107     mask = (df['用户ID'] == user_id) & \
108             (df['博主ID'] == up_id) & \
109             (df['时间'] >= start_period) & \
110             (df['时间'] < target_date) & \
111             (df['时间'].dt.weekday == target_weekday)
112     return len(df[mask])
113
114 # --- 预测函数：预测互动数最多的前三名博主 ---
115 def predict_top_interacted_ups(df: pd.DataFrame, target_user_ids: list, future_date_to_predict:
116     ↪ datetime,
117     top_n: int = 3) -> dict:
118     predictions = {}
119     print(f"\n===== 开始预测在 {future_date_to_predict.strftime('%Y-%m-%d')} 互动潜力最高的前 {top_n}
120     ↪ 名博主 =====")
121
122     effective_end_date_for_history = future_date_to_predict
123
124     for user_id in target_user_ids:
125         print(f"\n--- 处理用户: {user_id} ---")
126         user_data = df[df['用户ID'] == user_id]
127         if user_data.empty:
128             print(f" 用户 {user_id} 无历史数据。")
129             predictions[user_id] = []
130             continue

```

```

129 interacted_up_ids = user_data[user_data['博主ID'] != "UNKNOWN_BLOGGER"]['博主ID'].unique()
130 if not interacted_up_ids.any():
131     print(f" 用户 {user_id} 未与任何已知博主互动过。")
132     predictions[user_id] = []
133     continue
134
135 print(f" 用户 {user_id} 历史上互动过的博主 (候选): {list(interacted_up_ids)}")
136
137 up_scores = {}
138 for up_id in interacted_up_ids:
139     interactions_last_3_days = get_interactions_count(user_data, user_id, up_id,
140                                                         effective_end_date_for_history -
141                                                         ↪ timedelta(days=3),
142                                                         effective_end_date_for_history)
143     interactions_last_7_days = get_interactions_count(user_data, user_id, up_id,
144                                                         effective_end_date_for_history -
145                                                         ↪ timedelta(days=7),
146                                                         effective_end_date_for_history)
147     all_historical_interactions = get_interactions_count(user_data, user_id, up_id,
148                                                         user_data['时间'].min(),
149                                                         effective_end_date_for_history)
150     days_since_last_interaction = get_days_since_last_interaction_with_up(user_data, user_id,
151                                     ↪ up_id,
152                                     effective_end_date_for_history
153                                     ↪ )
154     interactions_same_weekday = get_interactions_on_same_weekday_with_up(user_data, user_id,
155                                     ↪ up_id,
156                                     future_date_to_predict
157                                     ↪ ,
158                                     4)
159
160     score = 0
161     score += interactions_last_3_days * 3.0
162     score += interactions_last_7_days * 1.5
163     if all_historical_interactions > 0: score += np.log1p(all_historical_interactions) * 1.0
164     if days_since_last_interaction <= 1:
165         score += 5.0
166     elif days_since_last_interaction <= 3:
167         score += 3.0
168     elif days_since_last_interaction <= 7:
169         score += 1.0
170     score += interactions_same_weekday * 2.0
171     up_scores[up_id] = score
172
173 if not up_scores:
174     print(f" 用户 {user_id} 没有可计算得分的博主。")
175     predictions[user_id] = []
176 else:
177     sorted_ups_with_scores = sorted(up_scores.items(), key=lambda item: item[1], reverse=True)
178     top_ups_predicted = [up_info for up_info in sorted_ups_with_scores[:top_n]]
179
180     predictions[user_id] = top_ups_predicted
181     if top_ups_predicted:
182         print(
183             f" >> 预测用户 {user_id} 在 {future_date_to_predict.strftime('%Y-%m-%d')} 互动潜力
184             ↪ 最高的前 {top_n} 名博主:")

```

```

177         for rank, (up_id_pred, score_pred) in enumerate(top_ups_predicted, 1):
178             print(f"        {rank}. {up_id_pred} (潜力得分: {score_pred:.2f})")
179     else:
180         print(f" >> 用户 {user_id} 没有足够的博主进行Top {top_n} 预测。")
181     return predictions
182
183
184 # --- 主执行部分 ---
185 if __name__ == "__main__":
186     try:
187         df_main = load_data()
188
189         if df_main is None or df_main.empty:
190             print("数据加载失败或数据为空，无法进行预测。")
191         else:
192             # ***** 修改这里：添加 U9 和 U22405 *****
193             users_to_predict_interaction = ['U16', 'U48420', 'U9', 'U22405']
194             date_to_predict_interaction_str = "2024-07-23"
195             future_interaction_date = datetime.strptime(date_to_predict_interaction_str, "%Y-%m-%d")
196
197             actual_users_in_df = df_main['用户ID'].unique()
198             users_to_process = [u for u in users_to_predict_interaction if u in actual_users_in_df]
199             missing_users = [u for u in users_to_predict_interaction if u not in actual_users_in_df]
200
201             if missing_users:
202                 print(f"警告：以下指定的用户ID在加载的数据中未找到，将不会为他们进行预测：{
203                     ↪ missing_users}")
204
205             if not users_to_process:
206                 print("错误：所有指定的用户ID均未在加载数据中找到。")
207             else:
208                 interaction_predictions_top3 = predict_top_interacted_ups(df_main, users_to_process,
209                                     future_interaction_date,
210                                     ↪ top_n=3)
211
212                 print(f"\n--- {date_to_predict_interaction_str} 互动潜力最高博主 (Top 3) 预测结果 ---")
213                 for user, top_up_list in interaction_predictions_top3.items():
214                     if top_up_list:
215                         print(f"用户 {user} 在 {date_to_predict_interaction_str} 预计互动潜力最高的前3
216                             ↪ 名博主:")
217                         for i, (up_id_res, score_res) in enumerate(top_up_list, 1):
218                             print(f"    {i}. 博主: {up_id_res} (潜力得分: {score_res:.2f})")
219                     elif user in users_to_process: # 确保是处理过但没有结果的用户
220                         print(
221                             f"用户 {user} 在 {date_to_predict_interaction_str} 无法预测互动潜力最高的博
222                             ↪ 主 (可能无足够互动历史或候选)。")
223
224                 # 打印在数据中未找到的用户
225                 for missing_user in missing_users:
226                     # 检查是否是因为未在user_files_to_load中定义导致未加载
227                     if missing_user not in [k for k, v in user_files_info.items() if
228                                             v is not None]: # 假设user_files_info在load_data中定义
229                         print(f"用户 {missing_user} 的数据文件未在加载列表中定义，无预测结果。")
230                     else: # 否则就是加载了但数据为空或不符合要求
231                         print(f"用户 {missing_user} 未在数据中找到或数据不符合要求，无预测结果。")

```

```

228
229     except FileNotFoundError as fnf_error:
230         print(f"程序终止：必要文件未找到。{fnf_error}")
231     except ValueError as val_error:
232         print(f"程序终止：数据问题。{val_error}")
233     except Exception as e:
234         print(f"程序执行过程中发生未预料的错误：{e}")
235         import traceback
236
237         traceback.print_exc()

```

10.8 输入保存系统

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from typing import Dict, Optional, Union
5 INTEREST_INCREMENTS: Dict[int, float] = {
6     1: 0.5,
7     2: 2.0,
8     3: 3.0,
9 }
10 FOLLOW_ACTION_CODE: int = 4
11 FOLLOW_INTEREST_LEVEL: float = 15.0
12 DAILY_DECAY_AMOUNT: float = 0.5
13 DEFAULT_INCREMENT: float = 0.0
14 def load_data(c1_path: str = 'C1数据.csv', c2_path: str = 'C2.csv') -> pd.DataFrame:
15
16     try:
17         df1 = pd.read_csv(c1_path, parse_dates=['时间'], encoding='utf-8-sig')
18         df2 = pd.read_csv(c2_path, parse_dates=['时间'], encoding='utf-8-sig')
19         print(f"从 {c1_path} 和 {c2_path} 加载数据成功。")
20
21     except FileNotFoundError as e:
22         print(f"错误：{e}。请确保CSV文件 '{c1_path}' 和 '{c2_path}' 与脚本在同一目录下，或者提供正确路
           ↳ 径。")
23         raise
24     except ValueError as e:
25         if "Missing column provided to 'parse_dates'" in str(e) or "'时间' not in columns" in str(
26             e) or "could not be parsed" in str(e):
27             print(
28                 f"读取CSV文件时出错：找不到名为 '时间' 的列或 '时间' 列无法被解析为日期。请检查CSV文件
           ↳ 中的列名和日期格式是否正确，并与代码中的 '时间' 匹配。")
29             try:
30                 temp_df1_cols = pd.read_csv(c1_path, encoding='utf-8-sig', nrows=0).columns.tolist()
31                 print(f"文件 {c1_path} 的侦测到的列名：{temp_df1_cols}")
32             except Exception as read_err:
33                 print(f"尝试读取 {c1_path} 列名失败：{read_err}")
34             try:
35                 temp_df2_cols = pd.read_csv(c2_path, encoding='utf-8-sig', nrows=0).columns.tolist()
36                 print(f"文件 {c2_path} 的侦测到的列名：{temp_df2_cols}")
37             except Exception as read_err:
38                 print(f"尝试读取 {c2_path} 列名失败：{read_err}")
39         else:
40             print(f"读取CSV文件时发生值错误：{e}")

```

```

41         raise
42     except Exception as e:
43         print(f"读取CSV文件时发生未知错误: {e}")
44         raise
45
46     required_cols = ['用户ID', '博主ID', '时间', '用户行为']
47     for df_check, name in [(df1, c1_path), (df2, c2_path)]:
48         # 去除列名中可能存在的前后空格
49         df_check.columns = df_check.columns.str.strip()
50         for col in required_cols:
51             if col not in df_check.columns:
52                 actual_cols_message = f"文件 {name} 的实际列名为: {list(df_check.columns)}" if not
53                     ↪ df_check.empty else f"文件 {name} 为空或无法读取列名。"
54                 raise ValueError(
55                     f"文件 {name} 中缺少必需的列 '{col}' (期望的列名列表是 {required_cols})。\\n{
56                         ↪ actual_cols_message}")
57
58     df1['用户ID'] = df1['用户ID'].astype(str)
59     df1['博主ID'] = df1['博主ID'].astype(str)
60     df2['用户ID'] = df2['用户ID'].astype(str)
61     df2['博主ID'] = df2['博主ID'].astype(str)
62
63     df_all = pd.concat([df1, df2], ignore_index=True)
64     return df_all
65
66 # --- 新的兴趣度计算函数 ---
67 def compute_interest_series_new_model(
68     df: pd.DataFrame,
69     user_id: str, # 明确ID为字符串类型
70     up_id: str, # 明确ID为字符串类型
71     start_time_requested: Optional[pd.Timestamp] = None,
72     end_time_requested: Optional[pd.Timestamp] = None,
73     freq: str = '1H' # 输出频率
74 ) -> pd.Series:
75     """
76     根据新的状态模型计算兴趣度。
77     """
78     print(f"compute_interest_series_new_model: 正在为用户 '{user_id}', 博主 '{up_id}' 筛选数据...")
79     # 确保DataFrame中的ID列也是字符串类型, 以便正确比较
80     # (已在load_data中处理, 但双重检查无害或在此处再次转换)
81     # df['用户ID'] = df['用户ID'].astype(str)
82     # df['博主ID'] = df['博主ID'].astype(str)
83
84     subset = df[(df['用户ID'] == user_id) & (df['博主ID'] == up_id)].sort_values(by='时间')
85     print(f"compute_interest_series_new_model: 筛选后数据条数: {len(subset)}")
86
87     if subset.empty:
88         print(f"未找到用户 '{user_id}' 和博主 '{up_id}' 之间的交互记录。")
89         if start_time_requested and end_time_requested:
90             time_index_empty = pd.date_range(start=start_time_requested, end=end_time_requested, freq=
91                 ↪ freq)
92             if time_index_empty.empty and start_time_requested == end_time_requested:
93                 time_index_empty = pd.date_range(start=start_time_requested, periods=1, freq=freq)
94             return pd.Series(0.0, index=time_index_empty, name='Interest')

```

```

93     return pd.Series(dtype=float, name='Interest')
94
95     sim_start_time = start_time_requested if start_time_requested is not None else subset['时间'].min()
96     sim_end_time = end_time_requested if end_time_requested is not None else subset['时间'].max()
97
98     if sim_start_time == sim_end_time:
99         sim_end_time = sim_start_time + pd.to_timedelta(freq)
100
101     effective_start_for_range = sim_start_time
102     effective_end_for_range = sim_end_time
103
104     output_time_index = pd.date_range(start=effective_start_for_range, end=effective_end_for_range,
105         ↪ freq=freq)
106
107     if output_time_index.empty:
108         if effective_start_for_range == effective_end_for_range:
109             output_time_index = pd.date_range(start=effective_start_for_range, periods=1, freq=freq)
110
111     if output_time_index.empty:
112         print(
113             f"警告: 生成的 output_time_index 为空。起始: {effective_start_for_range}, 结束: {
114                 ↪ effective_end_for_range}, 频率: {freq}")
115         return pd.Series(dtype=float, name='Interest')
116
117     final_values = np.zeros(len(output_time_index))
118     current_interest_final: float = 0.0
119     has_followed_final: bool = False
120     last_decay_date_final: Optional[pd.Timestamp.date] = None
121
122     actual_follow_time: Optional[pd.Timestamp] = None
123     follow_events_subset = subset[subset['用户行为'] == FOLLOW_ACTION_CODE]
124     if not follow_events_subset.empty:
125         actual_follow_time = follow_events_subset['时间'].min()
126         print(f"检测到关注行为, 首次关注时间: {actual_follow_time}")
127
128     for i, t_point in enumerate(output_time_index):
129         t_point_date = t_point.date()
130
131         # 1. 基于 actual_follow_time 检查是否已进入“已关注”状态
132         if not has_followed_final and actual_follow_time is not None and t_point >= actual_follow_time:
133             current_interest_final = FOLLOW_INTEREST_LEVEL
134             has_followed_final = True
135
136         # 2. 如果已关注, 则兴趣度固定
137         if has_followed_final:
138             current_interest_final = FOLLOW_INTEREST_LEVEL
139
140         # 3. 如果未关注且兴趣度大于0, 则进行衰减
141         elif current_interest_final > 0:
142             if last_decay_date_final is not None:
143                 days_passed_since_last_decay = (t_point_date - last_decay_date_final).days
144                 if days_passed_since_last_decay > 0:
145                     decay_amount = DAILY_DECAY_AMOUNT * days_passed_since_last_decay
146                     current_interest_final = max(0, current_interest_final - decay_amount)
147                     print(f"时间点 {t_point}: 衰减 {decay_amount}, 衰减后兴趣度: {
148                         ↪ current_interest_final}")

```

```

145     # 无论是否衰减, 都更新上次衰减检查的日期
146     last_decay_date_final = t_point_date
147
148     # 4. 处理当前时间点 [interval_start, t_point] 内的事件
149     # interval_start应该是上一个output_time_index的时间点, 或者是序列开始前的某个点
150     # t_point是当前output_time_index的时间点
151     # 我们要找的是在 (上一个t_point, 当前t_point] 区间的事件
152     # 对于第一个点, 区间是 (-inf, 第一个t_point]
153     interval_start_event_check = output_time_index[i - 1] if i > 0 else (
154         t_point - pd.to_timedelta(freq)) # 更合理的区间开始
155     # 或者, 对于更精确的事件聚合:
156     # interval_start_event_check = pd.Timestamp.min if i == 0 else output_time_index[i-1]
157
158     events_in_interval = subset[(subset['时间'] > interval_start_event_check) & (subset['时间'] <=
159         ↪ t_point)]
160
161     if not events_in_interval.empty:
162         for _, event in events_in_interval.iterrows():
163             # 再次检查事件是否触发关注状态 (以防事件时间正好在关注时间点或之后)
164             if not has_followed_final and actual_follow_time is not None and event['时间'] >=
165                 ↪ actual_follow_time:
166                 current_interest_final = FOLLOW_INTEREST_LEVEL
167                 has_followed_final = True
168                 print(f"事件 {event['时间']}: 触发关注状态, 兴趣度设为 {current_interest_final}")
169
170             if has_followed_final: # 如果在处理事件过程中变为已关注
171                 current_interest_final = FOLLOW_INTEREST_LEVEL
172             elif event['用户行为'] == FOLLOW_ACTION_CODE:
173                 current_interest_final = FOLLOW_INTEREST_LEVEL
174                 has_followed_final = True
175                 actual_follow_time = event['时间'] # 更新实际关注时间, 以防有多个关注事件
176                 print(f"事件 {event['时间']}: 用户行为关注, 兴趣度设为 {current_interest_final}")
177             else: # 其他非关注行为, 增加兴趣度
178                 increment = INTEREST_INCREMENTS.get(event['用户行为'], DEFAULT_INCREMENT)
179                 current_interest_final += increment
180                 current_interest_final = max(0, current_interest_final) # 兴趣度不能为负
181                 print(
182                     f"事件 {event['时间']}: 行为 {event['用户行为']}, 增量 {increment}, 当前兴趣度
183                     ↪ {current_interest_final}")
184
185             final_values[i] = current_interest_final
186
187     # 如果在当前时间点因为事件进入关注状态, 则不再需要衰减日期跟踪
188     if not has_followed_final:
189         last_decay_date_final = t_point_date # 更新衰减日期跟踪
190
191     final_series = pd.Series(final_values, index=output_time_index, name="Interest")
192
193     if start_time_requested is not None:
194         final_series = final_series[final_series.index >= start_time_requested]
195     if end_time_requested is not None:
196         final_series = final_series[final_series.index <= end_time_requested]
197
198     print(f"compute_interest_series_new_model: 计算完成. 最终序列长度: {len(final_series)}")
199     return final_series

```



```

197
198 # --- 绘图函数 ---
199 def plot_interest_new_model(
200     user_id_to_plot: str, # 明确ID为字符串类型
201     up_id_to_plot: str, # 明确ID为字符串类型
202     df_all: Optional[pd.DataFrame] = None,
203     start_time_str_input: Optional[str] = None,
204     end_time_str_input: Optional[str] = None,
205     freq_input: str = '1H'
206 ) -> None:
207     """
208     加载数据（如果未提供），计算并绘制用户对博主的兴趣度（新模型）。
209     """
210     if df_all is None:
211         print("plot_interest_new_model: df_all 为空，正在加载数据...")
212         df_all = load_data() # 默认路径
213     else:
214         print("plot_interest_new_model: df_all 已提供。")
215
216     parsed_start_time = None
217     if start_time_str_input and start_time_str_input.strip():
218         try:
219             parsed_start_time = pd.to_datetime(start_time_str_input)
220         except Exception as e:
221             print(f"警告：无法解析开始时间 '{start_time_str_input}'，将自动确定开始时间。错误：{e}")
222
223     parsed_end_time = None
224     if end_time_str_input and end_time_str_input.strip():
225         try:
226             parsed_end_time = pd.to_datetime(end_time_str_input)
227         except Exception as e:
228             print(f"警告：无法解析结束时间 '{end_time_str_input}'，将自动确定结束时间。错误：{e}")
229
230     print(f"plot_interest_new_model: 尝试为用户 '{user_id_to_plot}'，博主 '{up_id_to_plot}' 计算兴趣度"
231           ↪ )
232     print(
233         f"plot_interest_new_model: 请求开始时间: {parsed_start_time}, 请求结束时间: {parsed_end_time},
234         ↪ 频率: {freq_input}")
235
236     try:
237         interest_series = compute_interest_series_new_model(
238             df_all,
239             user_id_to_plot,
240             up_id_to_plot,
241             start_time_requested=parsed_start_time,
242             end_time_requested=parsed_end_time,
243             freq=freq_input
244         )
245         print(f"plot_interest_new_model: 兴趣度序列计算完成。长度: {len(interest_series)}")
246         if not interest_series.empty:
247             print(f"plot_interest_new_model: 计算得到的兴趣度序列（前5条数据，如果存在）:\n{
248                 ↪ interest_series.head()}")
249             print(f"plot_interest_new_model: 计算得到的兴趣度序列（后5条数据，如果存在）:\n{
250                 ↪ interest_series.tail()}")
251         else:

```

```

248         print("plot_interest_new_model: 计算得到的兴趣度序列为空。")
249
250     except ValueError as e:
251         print(f"计算兴趣度时出错 (在 plot_interest_new_model 中): {e}")
252         return
253     except Exception as e:
254         import traceback
255         print(f"计算兴趣度时发生未预料的错误 (在 plot_interest_new_model 中): {e}")
256         traceback.print_exc()
257         return
258
259     if interest_series.empty:
260         print(f"没有可为用户 '{user_id_to_plot}' 和博主 '{up_id_to_plot}' 绘制的兴趣数据。(序列为空)")
261         # 可以选择在这里创建一个空的图表或者直接返回
262         plt.figure(figsize=(12, 6))
263         plt.title(f"用户 '{user_id_to_plot}' 对博主 '{up_id_to_plot}' 的兴趣度 (无数据)", fontsize=14)
264         plt.xlabel('时间', fontsize=12)
265         plt.ylabel('兴趣度水平', fontsize=12)
266         plt.text(0.5, 0.5, '无交互数据', horizontalalignment='center', verticalalignment='center',
267                 transform=plt.gca().transAxes)
268         plt.grid(True, linestyle='--', alpha=0.7)
269         plt.tight_layout()
270         plt.show()
271         return
272
273     print("plot_interest_new_model: 准备绘图...")
274     plt.figure(figsize=(12, 6))
275     plt.step(interest_series.index, interest_series.values, where='post', marker='.', linestyle='-')
276     plt.title(f"用户 '{user_id_to_plot}' 对博主 '{up_id_to_plot}' 的兴趣度 (状态模型)", fontsize=14)
277     plt.xlabel('时间', fontsize=12)
278     plt.ylabel('兴趣度水平', fontsize=12)
279     plt.xticks(rotation=45)
280     plt.grid(True, linestyle='--', alpha=0.7)
281     plt.tight_layout()
282     # plt.savefig(f'interest_plot_{user_id_to_plot}_{up_id_to_plot}.png') # 动态文件名
283     # print(f"plot_interest_new_model: 图片已保存到 interest_plot_{user_id_to_plot}_{up_id_to_plot}.png
284     ↪ ")
285     print("plot_interest_new_model: 尝试显示 plt.show()...")
286     plt.show()
287     print("plot_interest_new_model: plt.show() 执行完毕。")
288
289 # --- 主程序执行部分 ---
290 if __name__ == '__main__':
291     c1_file_path = 'C1数据.csv'
292     c2_file_path = 'C2.csv' # 确保这个文件存在, 或者根据需要调整逻辑
293     default_freq = '1H'
294
295     print("--- 用户兴趣度分析程序 (状态模型) ---")
296
297     # 获取用户输入, 并直接作为字符串处理
298     user_id_input_str = input("请输入用户ID (例如 U1, U2642): ").strip()
299     up_id_input_str = input("请输入博主ID (例如 B60, B123): ").strip()
300
301     # ID 参数将作为字符串传递

```

```

302 user_id_arg = user_id_input_str
303 up_id_arg = up_id_input_str
304
305 start_time_input = input(
306     f"请输入开始时间 (格式 YYYY-MM-DD HH:MM 或 YYYY-MM-DD, 直接回车则根据数据自动确定): ").strip()
307 end_time_input = input(
308     f"请输入结束时间 (格式 YYYY-MM-DD HH:MM 或 YYYY-MM-DD, 直接回车则根据数据自动确定): ").strip()
309 freq_input_val = input(
310     f"请输入时间频率 (例如 '1H', '1D', '30min', 直接回车使用默认 '{default_freq}'): ").strip() or
    ↪ default_freq
311
312 try:
313     print(f"准备从 {c1_file_path} 和 {c2_file_path} 加载数据...")
314     df_loaded = load_data(c1_path=c1_file_path, c2_path=c2_file_path)
315     print("--- 加载的DataFrame信息 ---")
316     print("列名:", df_loaded.columns.tolist())
317     print("数据类型:\n", df_loaded.dtypes)
318     print("-----")
319     print(
320         f"准备为用户: '{user_id_arg}' (类型: {type(user_id_arg)}), 博主: '{up_id_arg}' (类型: {type
    ↪ (up_id_arg)}) 绘制兴趣度图")
321     plot_interest_new_model(
322         user_id_arg,
323         up_id_arg,
324         df_all=df_loaded,
325         start_time_str_input=start_time_input if start_time_input else None,
326         end_time_str_input=end_time_input if end_time_input else None,
327         freq_input=freq_input_val
328     )
329 except FileNotFoundError:
330     print("退出程序: 一个或多个数据文件未找到。请检查文件名和路径。")
331 except ValueError as ve:
332     print(f"发生值错误 (可能是数据格式、列名问题或ID类型不匹配): {ve}")
333     import traceback
334
335     traceback.print_exc()
336 except Exception as e:
337     import traceback
338
339     print(f"发生未预料到的错误: {e}")
340     traceback.print_exc()
341
342 print("--- 程序执行完毕 ---")

```

10.9 问题三的兴趣判定

```

1 import pandas as pd
2 import numpy as np
3 from datetime import datetime, timedelta
4
5
6 # --- load_data 函数 (与您提供的版本一致, 用于加载所有历史数据) ---
7 def load_data():
8     print("开始加载数据...")
9     # C1数据: 7.11-7.20 (所有用户历史数据)

```

```

10     try:
11         df_c1 = pd.read_csv('C1数据.csv', parse_dates=['时间'], encoding='utf-8-sig')
12         print("C1数据.csv 加载成功。")
13     except FileNotFoundError:
14         print("错误: C1数据.csv 未找到。此文件对于历史分析至关重要。")
15         df_c1 = pd.DataFrame() # 确保即使文件不存在也能继续 (如果特定用户文件已包含足够历史)
16         # raise # 或者如果C1是绝对必要的, 则取消注释raise
17     except Exception as e:
18         print(f"加载 C1数据.csv 时出错: {e}")
19         df_c1 = pd.DataFrame()
20
21     # C2数据: 7.22 (所有用户当天数据)
22     try:
23         df_c2 = pd.read_csv('C2.csv', parse_dates=['时间'], encoding='utf-8-sig')
24         print("C2.csv 加载成功。")
25     except FileNotFoundError:
26         print("错误: C2.csv 未找到。")
27         df_c2 = pd.DataFrame()
28         # raise
29     except Exception as e:
30         print(f"加载 C2.csv 时出错: {e}")
31         df_c2 = pd.DataFrame()
32
33     # 用户特定文件加载逻辑 (假设您会补充U16.xlsx, U48420.xlsx等)
34     user_files_info = {
35         'U7': 'U7.xlsx',
36         'U52010': 'U52010.xlsx',
37         'U5769': 'U5769.xlsx',
38         'U14990': 'U14990.xlsx',
39         'U6749': 'U6749.xlsx',
40         'U16': 'U16.xlsx', # 添加U16
41         'U48420': 'U48420.xlsx' # 添加U48420
42     }
43     df_user_specific_list = []
44     for user_id_key, filename in user_files_info.items():
45         try:
46             df_user = pd.read_excel(filename, parse_dates=['时间'])
47             # 确保用户ID列存在, 如果不存在则从文件名添加
48             if '用户ID' not in df_user.columns:
49                 df_user['用户ID'] = user_id_key
50             else:
51                 df_user['用户ID'] = df_user['用户ID'].astype(str) # 统一为字符串
52
53             # 确保其他必要列存在, 如果不存在则创建空列或填充默认值
54             base_required_cols = ['时间', '用户ID']
55             optional_cols = ['用户行为', '博主ID']
56             for col in base_required_cols:
57                 if col not in df_user.columns:
58                     print(f"错误: 文件 {filename} 缺少核心列 '{col}'。跳过此文件。")
59                     raise ValueError(f"文件 {filename} 缺少核心列 '{col}'")
60             for col in optional_cols:
61                 if col not in df_user.columns:
62                     if col == '用户行为':
63                         df_user[col] = 0
64                     elif col == '博主ID':

```

```

65         df_user[col] = "UNKNOWN_BLOGGER"
66
67         df_user_specific_list.append(df_user)
68         print(f"{filename} 加载成功。")
69     except FileNotFoundError:
70         print(f"警告: {filename} 未找到。")
71     except Exception as e:
72         print(f"加载 {filename} 时出错: {e}")
73
74 all_dfs = [df_c1, df_c2] + df_user_specific_list
75 valid_dfs = [df for df in all_dfs if not df.empty] # 简化检查, 后续会检查必需列
76
77 if not valid_dfs:
78     print("错误: 没有成功加载任何数据文件。程序无法继续。")
79     raise ValueError("无有效数据加载")
80
81 df_combined = pd.concat(valid_dfs, ignore_index=True)
82 print(f"数据合并完成。总记录数: {len(df_combined)}")
83
84 # 在进行任何操作前, 确保核心列存在
85 required_cols_for_ops = ['用户ID', '时间']
86 if not all(col in df_combined.columns for col in required_cols_for_ops):
87     missing_cols = [col for col in required_cols_for_ops if col not in df_combined.columns]
88     print(f"错误: 合并后的DataFrame缺少以下核心列: {missing_cols}")
89     raise ValueError(f"合并后的DataFrame缺少核心列: {missing_cols}")
90
91 # 预处理, 确保 '用户行为' 和 '博主ID' 即使部分文件缺失也能安全处理
92 df_combined['用户ID'] = df_combined['用户ID'].astype(str)
93 if '博主ID' in df_combined.columns:
94     df_combined['博主ID'] = df_combined['博主ID'].astype(str)
95 else:
96     df_combined['博主ID'] = "UNKNOWN_BLOGGER" # 如果完全没有博主ID列
97
98 if '用户行为' in df_combined.columns:
99     df_combined['用户行为'] = pd.to_numeric(df_combined['用户行为'], errors='coerce').fillna(0).
100         ↳ astype(int)
101 else:
102     df_combined['用户行为'] = 0 # 如果完全没有用户行为列
103
104 df_combined.dropna(subset=['时间'], inplace=True) # 时间列是必须的
105 df_combined.sort_values(by='时间', inplace=True)
106
107 print("数据预处理完成。")
108 return df_combined
109
110 # --- interest_calculate, deadline_interest, seeking_proportion_n_1, ---
111 # --- judge_first, judge_result 函数 (与您提供的版本一致) ---
112 def interest_calculate(df: pd.DataFrame, user_id: str, up_id: str, beginning: datetime, ending:
113     ↳ datetime) -> float:
114     mask = (df['用户ID'] == user_id) & \
115         (df['博主ID'] == up_id) & \
116         (df['时间'] >= beginning) & \
117         (df['时间'] < ending) & \
118         (df['用户行为'] != 4)

```

```

118 interactions = df[mask]
119 interest = 0.0
120 for _, row in interactions.iterrows():
121     if row['用户行为'] == 1:
122         interest += 0.5
123     elif row['用户行为'] == 2:
124         interest += 2.0
125     elif row['用户行为'] == 3:
126         interest += 3.0
127 return interest
128
129
130 def deadline_interest(df: pd.DataFrame, user_id: str) -> float:
131     follow_records = df[(df['用户ID'] == user_id) & (df['用户行为'] == 4)].copy()
132     if follow_records.empty: return 0.0
133     pre_follow_interests = []
134     min_global_time = df['时间'].min() if not (df.empty or df['时间'].isna().all()) else datetime.min
135     for _, follow_event in follow_records.iterrows():
136         up_id_followed = follow_event['博主ID']
137         time_of_follow = follow_event['时间']
138         interest_before_this_follow = interest_calculate(df, user_id, up_id_followed, min_global_time,
139             ↪ time_of_follow)
140         if interest_before_this_follow > 0:
141             pre_follow_interests.append(interest_before_this_follow)
142     if not pre_follow_interests: return 0.0
143     return min(pre_follow_interests) * 0.60 # 使用您之前调整的0.60
144
145 def seeking_proportion_n_1(df: pd.DataFrame, user_id: str, up_id: str, reference_date_for_A1: datetime)
146     ↪ -> int:
147     """
148     修改: A1的参考日期变为可配置, 用于预测未来。
149     对于预测7月23日, 我们将使用7月22日的的数据作为A1的参考。
150     """
151     # A1: 基于参考日期的兴趣值
152     A1 = interest_calculate(df, user_id, up_id, reference_date_for_A1, reference_date_for_A1 +
153         ↪ timedelta(days=1))
154
155     # A2, A3, A4 的日期基准保持不变 (7.18-7.20, 7.19,7.20,ref_A1, 7.15-7.20)
156     date_20 = datetime(2024, 7, 20)
157     date_19 = datetime(2024, 7, 19)
158     date_18 = datetime(2024, 7, 18)
159     date_15 = datetime(2024, 7, 15)
160
161     interests_for_A2 = [
162         interest_calculate(df, user_id, up_id, date_18, date_18 + timedelta(days=1)),
163         interest_calculate(df, user_id, up_id, date_19, date_19 + timedelta(days=1)),
164         interest_calculate(df, user_id, up_id, date_20, date_20 + timedelta(days=1))
165     ]
166
167     A2 = np.mean([i for i in interests_for_A2 if pd.notnull(i) and i > 0]) if any(
168         pd.notnull(i) and i > 0 for i in interests_for_A2) else 0.0
169
170     interests_for_A3 = [ # A3现在也应该基于历史数据来评估趋势, A1是预测日的潜力
171         interest_calculate(df, user_id, up_id, date_19, date_19 + timedelta(days=1)),
172         interest_calculate(df, user_id, up_id, date_20, date_20 + timedelta(days=1)),

```

```

170     interest_calculate(df, user_id, up_id, reference_date_for_A1, reference_date_for_A1 + timedelta
171         ↪ (days=1))
172     # A1保持为参考日
173 ]
174 A3 = np.mean([i for i in interests_for_A3 if pd.notnull(i) and i > 0]) if any(
175     pd.notnull(i) and i > 0 for i in interests_for_A3) else 0.0
176
177 interests_for_A4 = []
178 current_date = date_15
179 while current_date <= date_20: # A4 只计算到7月20日
180     interests_for_A4.append(interest_calculate(df, user_id, up_id, current_date, current_date +
181         ↪ timedelta(days=1)))
182     current_date += timedelta(days=1)
183 A4 = np.mean([i for i in interests_for_A4 if pd.notnull(i) and i > 0]) if any(
184     pd.notnull(i) and i > 0 for i in interests_for_A4) else 0.0
185
186 # 趋势判断条件保持不变, 但A1的含义是“近期兴趣潜力”
187 condition_met = False
188 if A1 > 0: # 近期潜力大于0
189     if A2 == 0 and A4 == 0: # 如果历史平均为0, 任何潜力都算趋势好
190         condition_met = True
191     elif A1 > A2 and (A3 > A2 or A2 > A4 or A3 > A4): # 潜力高于近期平均, 且趋势不差
192         condition_met = True
193     elif A1 > A2 and A2 == 0 and A4 > 0: # 近期平均为0但更早有, 现在潜力又上来了
194         condition_met = True
195 return 1 if condition_met else 0
196
197 def judge_first(df: pd.DataFrame, user_id: str, up_id: str, reference_date_for_interest: datetime) ->
198     ↪ int:
199     """
200     修改: 当日兴趣的参考日期变为可配置。
201     对于预测7月23日, 我们将使用7月22日的数据作为兴趣参考。
202     """
203     user_threshold = deadline_interest(df, user_id)
204     interest_on_ref_date = interest_calculate(df, user_id, up_id, reference_date_for_interest,
205         reference_date_for_interest + timedelta(days=1))
206
207     if interest_on_ref_date > 0 and interest_on_ref_date >= user_threshold:
208         return 1
209     else:
210         return 0
211
212 def judge_result_for_future(df: pd.DataFrame, user_id: str, up_id: str, future_target_date: datetime)
213     ↪ -> int:
214     """
215     用于预测未来日期的关注。
216     A1和当日兴趣的计算将基于 future_target_date 的前一天 (例如, 用7月22日数据预测7月23日)。
217     """
218     # 用目标日期的前一天作为计算“当日”兴趣和A1的参考基准
219     reference_date_for_calc = future_target_date - timedelta(days=1) # 即 7月22日
220
221     trend_positive = seeking_proportion_n_1(df, user_id, up_id, reference_date_for_calc)
222     threshold_met = judge_first(df, user_id, up_id, reference_date_for_calc)

```

```

221
222 # print(f" [FutureJudge] User:{user_id}, UP:{up_id}, Trend:{trend_positive}, ThresholdMet:{
    ↳ threshold_met}")
223 if trend_positive == 1 and threshold_met == 1:
224     return 1
225 else:
226     return 0
227
228
229 # --- 新的预测函数: 预测未来某天可能发生的关注 ---
230 def predict_future_follows(df: pd.DataFrame, target_user_ids: list, future_date_to_predict: datetime)
    ↳ -> dict:
231     predictions = {}
232     print(f"\n===== 开始预测在 {future_date_to_predict.strftime('%Y-%m-%d')} 可能发生的关注 =====")
233
234     for user_id in target_user_ids:
235         print(f"\n处理用户: {user_id}")
236
237         # 1. 找出该用户历史上互动过的所有UP主
238         interacted_ups_df = df[df['用户ID'] == user_id]
239         if interacted_ups_df.empty:
240             print(f" 用户 {user_id} 无历史互动记录。")
241             predictions[user_id] = []
242             continue
243
244         all_interacted_up_ids = interacted_ups_df['博主ID'].unique()
245
246         # 2. 找出该用户历史上已经关注过的UP主 (截止到 future_date_to_predict 的前一天)
247         # 因为我们是预测 future_date_to_predict 当天是否会发生 *新的* 关注
248         already_followed_ups_df = df[
249             (df['用户ID'] == user_id) &
250             (df['用户行为'] == 4) &
251             (df['时间'] < future_date_to_predict) # 关注行为发生在目标日期之前
252         ]
253         already_followed_up_ids = set(already_followed_ups_df['博主ID'].unique())
254
255         # 3. 确定候选UP主: 互动过但尚未关注的
256         candidate_up_ids = [up_id for up_id in all_interacted_up_ids if
257                             up_id not in already_followed_up_ids and up_id != "UNKNOWN_BLOGGER"]
258
259         if not candidate_up_ids:
260             print(f" 用户 {user_id} 没有互动过且尚未关注的UP主。")
261             predictions[user_id] = []
262             continue
263
264         print(f" 用户 {user_id} 的候选关注UP主 (互动过且未关注): {candidate_up_ids}")
265
266         predicted_ups_for_this_user = []
267         for up_id in candidate_up_ids:
268             # 使用新的 judge_result_for_future 进行判断
269             if judge_result_for_future(df, user_id, up_id, future_date_to_predict) == 1:
270                 predicted_ups_for_this_user.append(up_id)
271                 print(f" >> 预测用户 {user_id} 在 {future_date_to_predict.strftime('%Y-%m-%d')} 将关注
    ↳ UP主 {up_id}")
272

```



```

273     predictions[user_id] = predicted_ups_for_this_user
274
275     return predictions
276
277
278 # --- 主执行部分 ---
279 if __name__ == "__main__":
280     try:
281         df_main = load_data()
282
283         if df_main is None or df_main.empty:
284             print("数据加载失败或数据为空，无法进行预测。")
285         else:
286             # 指定要预测关注的用户和目标日期
287             users_for_future_follow_prediction = ['U16', 'U48420']
288             date_to_predict_follows_str = "2024-07-23"
289             future_prediction_date = datetime.strptime(date_to_predict_follows_str, "%Y-%m-%d")
290
291             # 确保这些用户的数据实际存在于df_main中
292             actual_users_in_df = df_main['用户ID'].unique()
293             users_to_process_future = [u for u in users_for_future_follow_prediction if u in
294                                     ↪ actual_users_in_df]
295             missing_users_future = [u for u in users_for_future_follow_prediction if u not in
296                                   ↪ actual_users_in_df]
297
298             if missing_users_future:
299                 print(
300                     f"警告：以下指定的用户ID在加载的数据中未找到，将不会为他们进行未来关注预测：{
301                     ↪ missing_users_future}")
302
303             if not users_to_process_future:
304                 print("错误：所有指定用于未来关注预测的用户ID均未在加载数据中找到。")
305             else:
306                 future_follow_predictions = predict_future_follows(df_main, users_to_process_future,
307                                                                     future_prediction_date)
308
309                 print(f"\n--- {date_to_predict_follows_str} 关注预测结果 ---")
310                 total_future_predicted_follows = 0
311                 for user, followed_ups in future_follow_predictions.items():
312                     if followed_ups:
313                         print(
314                             f"用户 {user} 预计将在 {date_to_predict_follows_str} 关注的UP主：{'', '.join
315                             ↪ (followed_ups)}")
316                         total_future_predicted_follows += len(followed_ups)
317                     else:
318                         print(f"用户 {user} 预计在 {date_to_predict_follows_str} 不会新增关注。")
319                 for missing_user in missing_users_future:
320                     print(f"用户 {missing_user} 未在数据中找到，无未来关注预测结果。")
321                 print(
322                     f"\n总共预测到 {total_future_predicted_follows} 条在 {date_to_predict_follows_str}
323                     ↪ 可能发生的新增关注。")
324
325     except FileNotFoundError as fnf_error:
326         print(f"程序终止：必要文件未找到。{fnf_error}")
327     except ValueError as val_error:

```

```
323     print(f"程序终止：数据问题。{val_error}")
324 except Exception as e:
325     print(f"程序执行过程中发生未预料的错误：{e}")
326     import traceback
327
328     traceback.print_exc()
```